Heap de Fibonacci e Árvores van Emde Boas

Igor Peixoto Rodrigues, Pedro Henrique de Araújo Carvalho, Mariana Kaori Yano, Rita Mariê Amaral, Julya Gonçalves Pires, Samuel Pianetti Cordeiro

Estrutura da Apresentação

1. Introdução
2. Objetivos
3. Contextualização
4. Heap Fibonacci
5. Árvoress van Emde Boas
6. Conclusão

Introdução

- Processamento eficiente de dados: aplicações dinâmicas exigem inserções, remoções e atualizações frequentes de elementos.

- Filas de prioridade: permitem organizar elementos por chave e recuperar aqueles de maior prioridade.

- Limitações das estruturas tradicionais: heaps binários e árvores balanceadas realizam operações principais em O(log n).

Introdução

- Heap de Fibonacci: reduz o custo amortizado de diminuição de chave para O(1).

- Árvore de van Emde Boas: explora chaves inteiras em um universo finito, oferecendo operações em O(log log U).

- Desafio prático: vantagens assintóticas nem sempre representam maior desempenho real, devido a fatores como uso de memória, organização dos ponteiros e cache.

Objetivos

- Comparar o Heap de Fibonacci e a Árvore vEB, confrontando garantias teóricas com o desempenho real.
- Formalizar as estruturas e analisar suas complexidades de tempo e consumo de espaço.
- Avaliar o comportamento prático em algoritmos de grafos, evidenciando o impacto do hardware.

Contextualização

- Escalonamento, simulações e roteamento exigem manter conjuntos ordenados dinâmicos com alta frequência de inserções, remoções e consultas de ordem (mínimo, predecessor e sucessor).

- A busca por estruturas alternativas para superar a limitação logarítmica das abordagens tradicionais.

- Duas Abordagens:
  Amortização e Chaves Comparáveis (Heap de Fibonacci)
  Restrição de Domínio e Decomposição Recursiva (Árvore vEB)

Heap de Fibonacci

Motivação

- Criação: proposto por Michael L. Fredman e Robert E. Tarjan (1987).
- Necessidade: o Dijkstra e o Prim fazem poucas extrações de mínimo, mas muitas reduções de chave. Nos heaps binários, o DECREASE-KEY custa O(log n). Isso dá Dijkstra em O((V + E) log V), e em grafos densos o DECREASE-KEY domina o custo.
- Amortização: a organização da estrutura é adiada até a extração do mínimo.



ROBERT TARJAN



MICHAEL L. FREDMAN

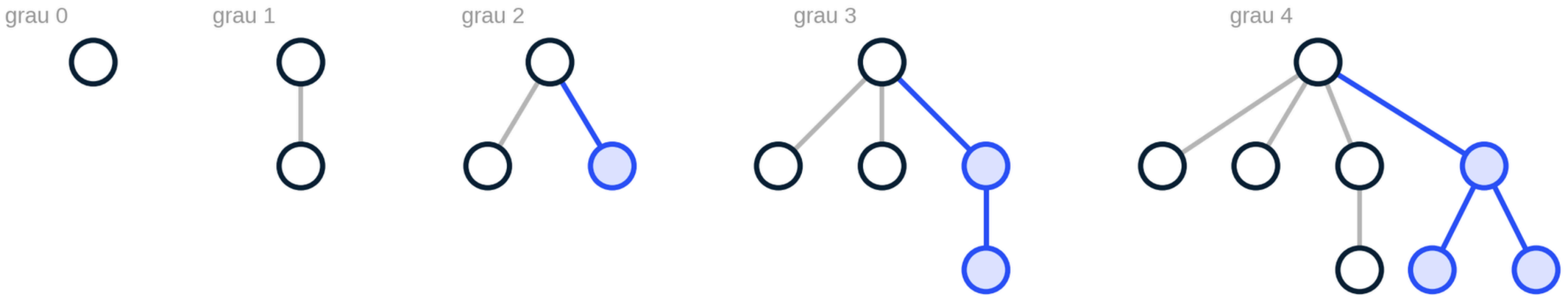
Motivação

DÁ PARA REDUZIR O DECREASE-KEY PARA O(1) SEM PIORAR AS OUTRAS OPERAÇÕES?

- Resultado: INSERT, UNION e DECREASE-KEY em O(1) amortizado, e EXTRACT-MIN em O(log n) amortizado. Isso leva o Dijkstra a O(E + V log V).

- Nome: vem dos números de Fibonacci, que aparecem na prova de que o grau máximo de um nó é O(log n).

árvores de tamanho mínimo: o número de nós de cada grau k é um número de Fibonacci, tam(k) = F(k+2)



grau 0	grau 1	grau 2	grau 3	grau 4
1 nó	**2 nós**	**3 nós**	**5 nós**	**8 nós**
$= F_2$	$= F_3$	$= F_4$	$= F_5$	$= F_6$
caso base	caso base	$2 + 1$	$3 + 2$	$5 + 3$
		$F_3 + F_2$	$F_4 + F_3$	$F_5 + F_4$

árvore do grau anterior (k − 1)

subárvore de grau k − 2, acrescentada como novo filho da raiz

Operações

Procedimentos básicos de uma fila de prioridade

Make-Heap, Insert, Minimum, Union, Extract-Min, Decrease-Key, Delete

Procedimentos auxiliares

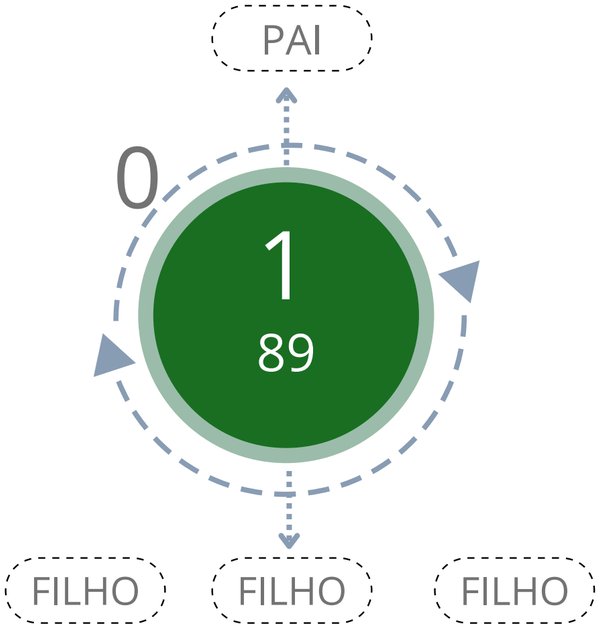
Consolidate, Heap-Link, Cut e Cascading-Cut

Pseudocódigos do Heap de Fibonacci

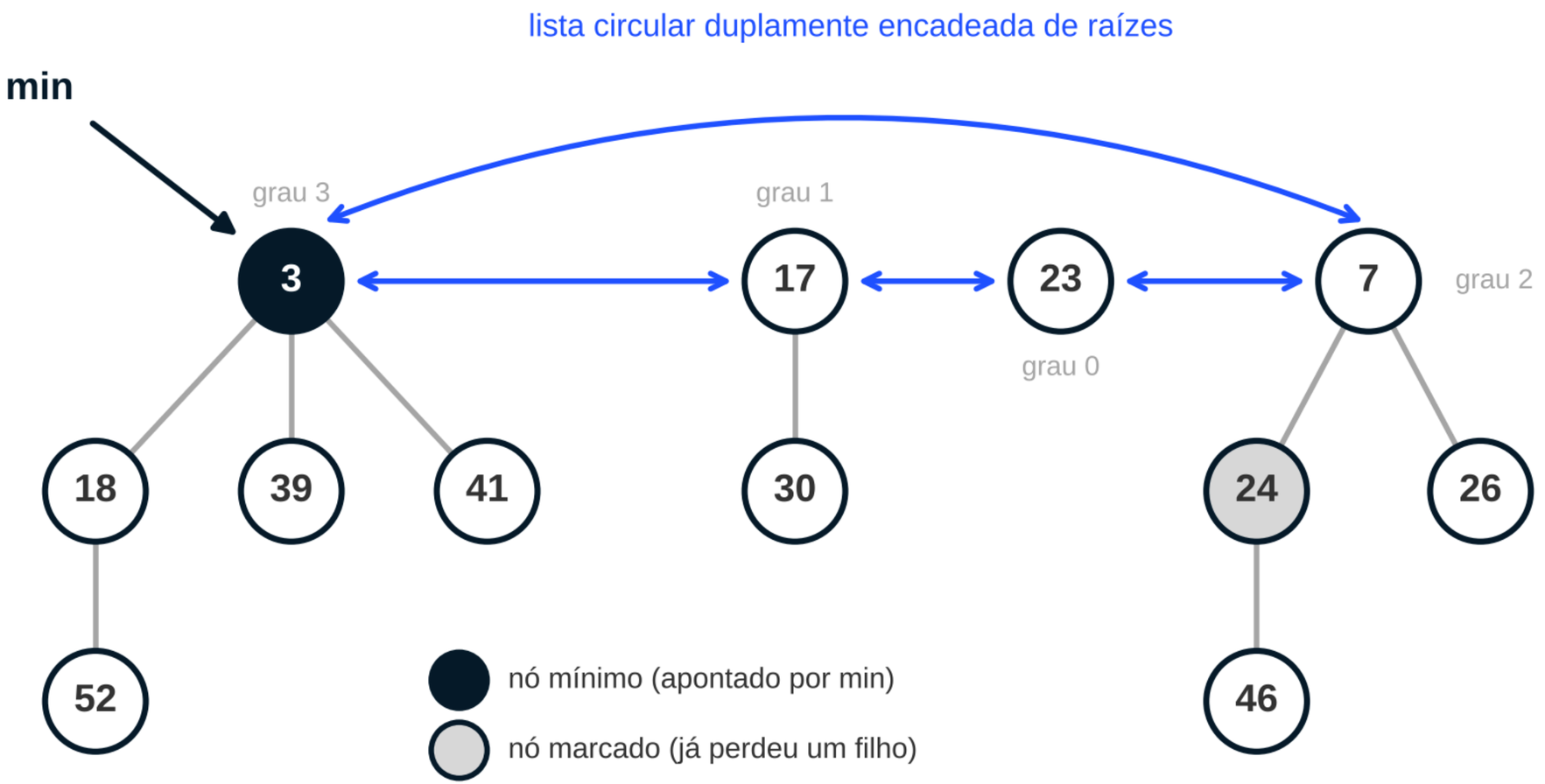
Estrutura do nó

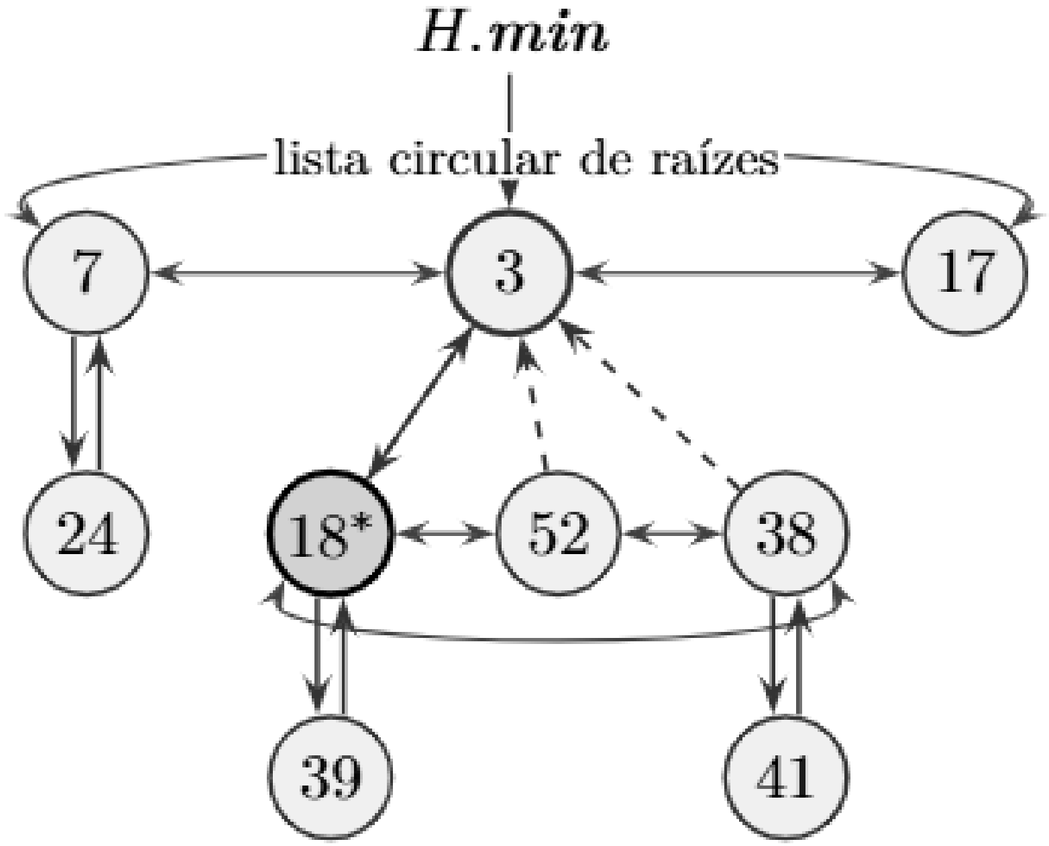
Nó do heap de Fibonacci

$x.chave$	chave do elemento (inteiro)
$x.valor$	valor associado, usado como identificador (inteiro)
$x.grau$	número de filhos de x
$x.marca$	TRUE se x perdeu um filho desde que foi ligado a outro nó
$x.pai$	ponteiro para o pai, ou NIL se x é raiz
$x.filho$	ponteiro para um dos filhos, ou NIL
$x.esq$, $x.dir$	vizinhos na lista circular duplamente encadeada



Heap H: $H.min$ (ponteiro para a raiz de menor chave) e $H.n$ (número de nós).


lista circular duplamente encadeada de raízes
min
grau 3
grau 1
grau 2
grau 0
3
17
23
7
18
39
41
30
24
26
52
46
nó mínimo (apontado por min)
nó marcado (já perdeu um filho)


H.min
lista circular de raízes
7
3
17
24
18*
52
38
39
41

Pseudocódigos do Heap de Fibonacci

Make-Heap

MAKE-HEAP()

Saída: heap vazio

1 $H.min \leftarrow$ NIL;

2 $H.n \leftarrow 0$;

3 **retorna** H;

Elaborado pelos autores com base na implementação do grupo.

Pseudocódigos do Heap de Fibonacci

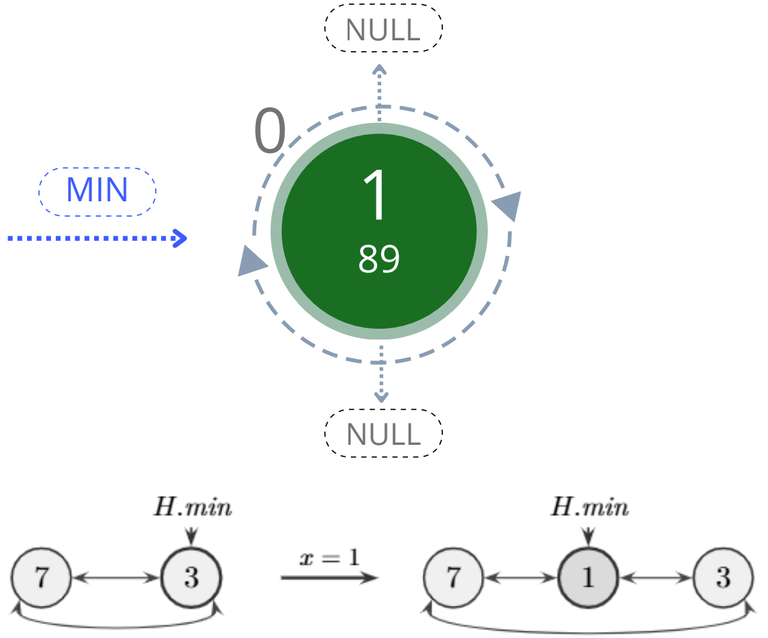
Insert e Minimum

INSERT(H, k, v)

Saída: ponteiro para o nó criado

```
1  x ← novo nó com x.chave ← k e x.valor ← v;
2  x.grau ← 0;  x.marca ← FALSE;
3  x.pai ← NIL;  x.filho ← NIL;
4  x.esq ← x;  x.dir ← x;
5  se H.min = NIL então
6  |   H.min ← x;
7  senão
8  |   INSERIRNALISTA(H.min, x);
9  |   se x.chave < H.min.chave então
10 |   |   H.min ← x;
11 H.n ← H.n + 1;
12 retorna x;
```

Elaborado pelos autores com base na implementação do grupo.

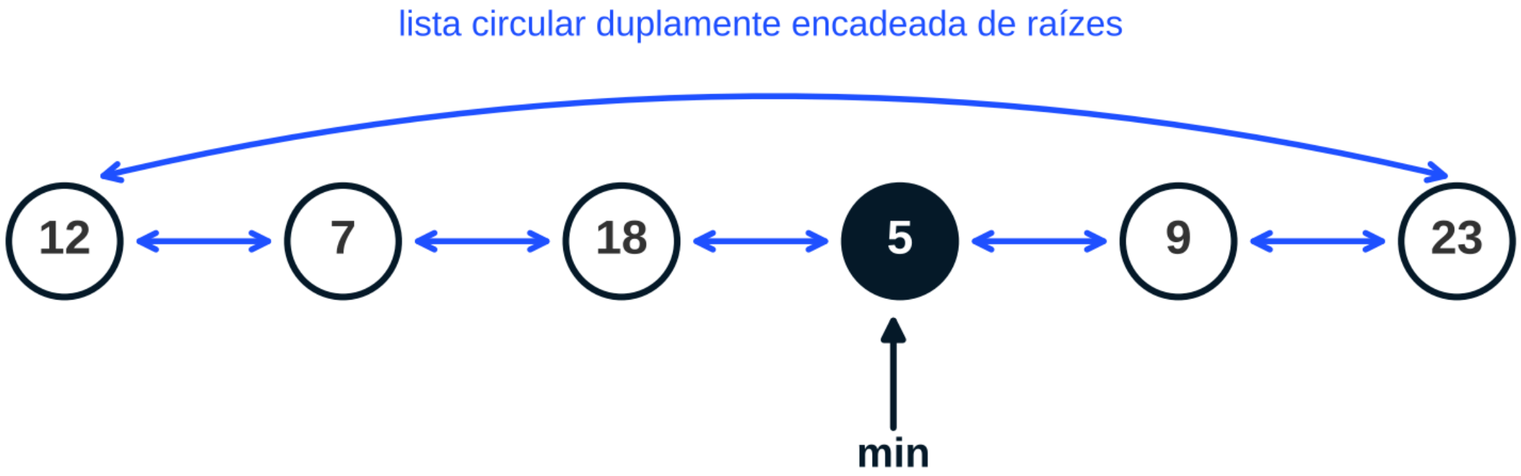


MINIMUM(H)

```
1  retorna H.min;
```

Elaborado pelos autores com base na implementação do grupo.

Insert e Minimum



Ordem de inserção: 12, 7, 18, 5, 9, 23 | todos os nós são raízes de grau 0

Pseudocódigos do Heap de Fibonacci

Union

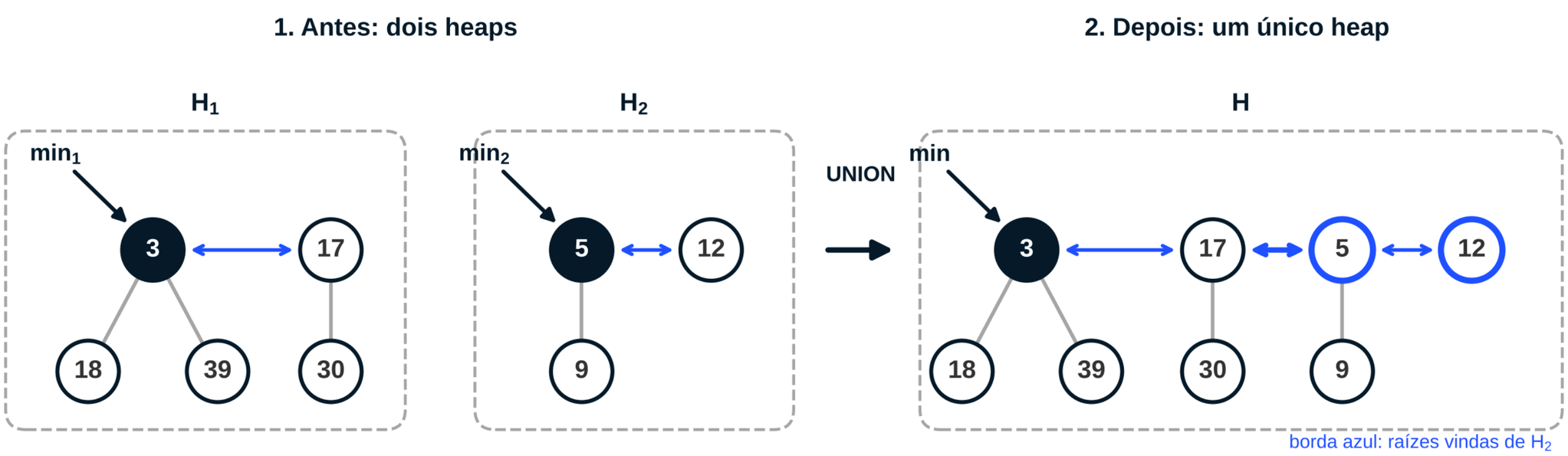
UNION(H_1, H_2)

Saída: heap H com os nós de H_1 e H_2

```
1  H ← MAKE-HEAP();
2  se H1.min = NIL então
3  |   H.min ← H2.min;
4  senão
5  |   H.min ← H1.min;
6  |   se H2.min ≠ NIL então
7  |   |   JUNTAR(H1.min, H2.min);
8  |   |   se H2.min.chave < H1.min.chave então
9  |   |   |   H.min ← H2.min;
10 H.n ← H1.n + H2.n;
11 esvaziar H1 e H2 (min ← NIL, n ← 0);
12 retorna H;
```

Elaborado pelos autores com base na implementação do grupo.

Union



Cada heap tem sua lista de raízes
e seu próprio min

As listas de raízes são concatenadas (ligação em azul) e o novo min é o menor entre min_1 e min_2 ($3 < 5$). Custo O(1), sem reorganização.

Pseudocódigos do Heap de Fibonacci

Extract-min, Consolidate e Heap-Link

EXTRACT-MIN(H)

Saída: nó de menor chave, ou NIL se o heap está vazio

```
1  z ← H.min;
2  se z = NIL então
3      retorna NIL;
4  para cada filho x de z faça
5      x.pai ← NIL;  x.marca ← FALSE;
6      INSERIRNALISTA(z, x) // promove x a raiz
7  REMOVERDALISTA(z);
8  se z = z.dir então
9      H.min ← NIL;
10 senão
11     H.min ← z.dir;
12     CONSOLIDATE(H);
13 H.n ← H.n − 1;
14 retorna z;
```

Elaborado pelos autores com base na implementação do grupo.

Pseudocódigos do Heap de Fibonacci

MÉTODOS AUXILIARES

CONSOLIDATE(H) (usado por EXTRACT-MIN)

```
1  R ← vetor com as raízes atuais de H;
2  A[0..⌊log_φ H.n⌋ + 1] ← NIL;
3  para cada w ∈ R faça
4      x ← w;  d ← x.grau;
5      enquanto A[d] ≠ NIL faça
6          y ← A[d];
7          se x.chave > y.chave então
8              trocar x ↔ y;
9          LINK(y, x);
10         A[d] ← NIL;  d ← d + 1;
11     A[d] ← x;
12 H.min ← NIL;
13 para cada i = 0 até ⌊log_φ H.n⌋ + 1 faça
14     se A[i] ≠ NIL então
15         reinserir A[i] na lista de raízes;
16         se H.min = NIL ou A[i].chave < H.min.chave então
17             H.min ← A[i];
```

Elaborado pelos autores com base na implementação do grupo.

LINK(y, x) (y torna-se filho de x)

```
1  REMOVERDALISTA(y);
2  y.pai ← x;  y.marca ← FALSE;
3  inserir y na lista de filhos de x;
4  x.grau ← x.grau + 1;
```

Elaborado pelos autores com base na implementação do grupo.

Extract-Min

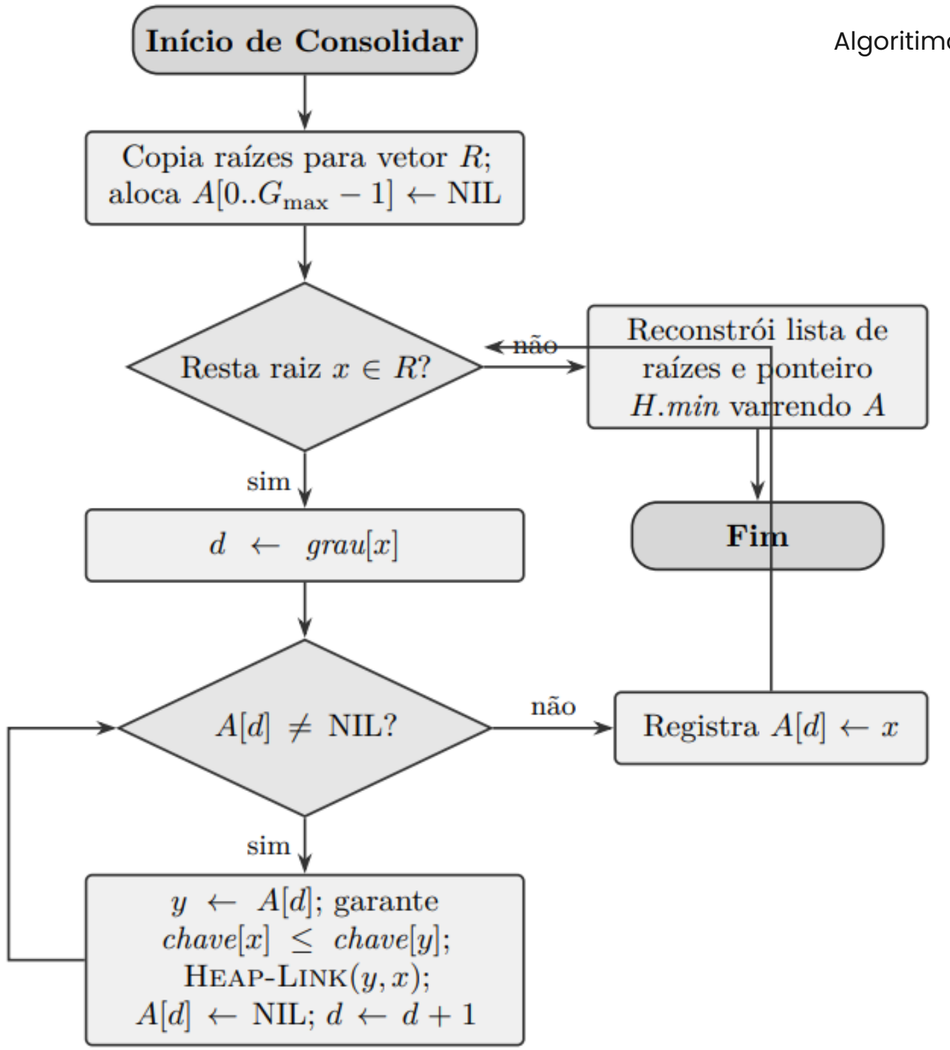
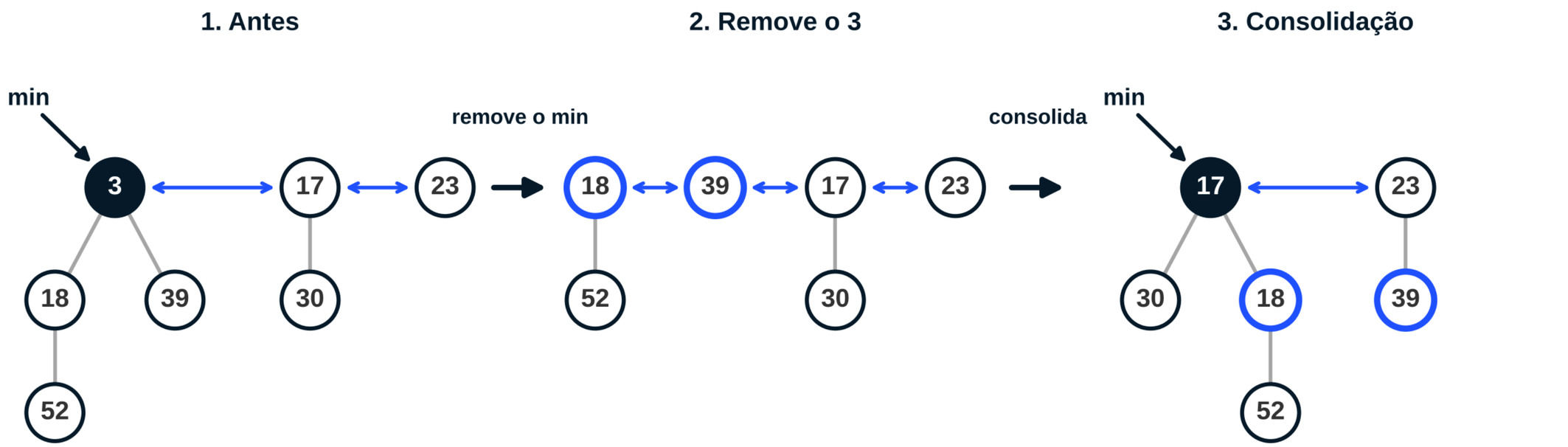


Figura 3 – Fluxograma iterativo do procedimento CONSOLIDAR: o laço interno resolve colisões de grau via HEAP-LINK até que cada posição $A[d]$ guarde no máximo uma árvore. Elaborado pelos autores com base na implementação do grupo.

Extract-Min



O mínimo é a raiz 3, apontada por min

Os filhos 18 e 39 viram raízes; min fica indefinido até a consolidação

Raízes de mesmo grau são unidas (17 < 18 e 23 < 39); o novo min é 17

Pseudocódigos do Heap de Fibonacci

Decrease-Key, Cascading-Cut e Cut

DECREASE-KEY(H, x, k)

```
1 se k > x.chave então
2     retorna;
3 x.chave ← k;  y ← x.pai;
4 se y ≠ NIL e x.chave < y.chave então
5     CUT(x, y);
6     CASCADING-CUT(y);
7 se x.chave < H.min.chave então
8     H.min ← x;
```

Elaborado pelos autores com base na implementação do grupo.

Pseudocódigos do Heap de Fibonacci

MÉTODOS AUXILIARES

CUT(x, y) (usado por DECREASE-KEY)

```
1 remover x da lista de filhos de y;
2 y.grau ← y.grau − 1;
3 x.pai ← NIL;  x.marca ← FALSE;
4 INSERIRNALISTA(H.min, x) // x passa a ser raiz
```

Elaborado pelos autores com base na implementação do grupo.

CASCADING-CUT(y) (usado por DECREASE-KEY)

```
1 z ← y.pai;
2 se z ≠ NIL então
3     se y.marca = FALSE então
4         y.marca ← TRUE;
5     senão
6         CUT(y, z);
7         CASCADING-CUT(z);
```

Elaborado pelos autores com base na implementação do grupo.

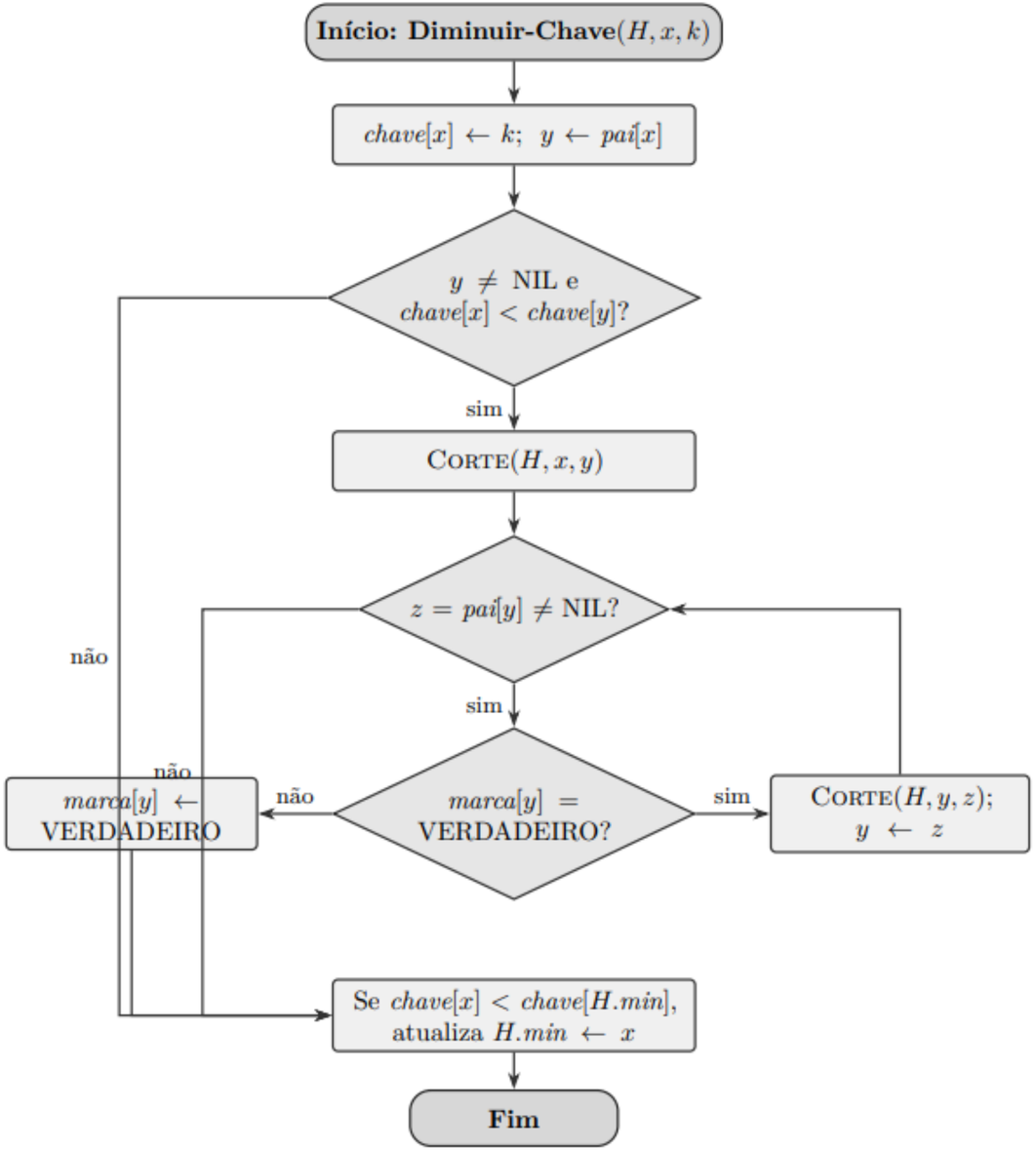
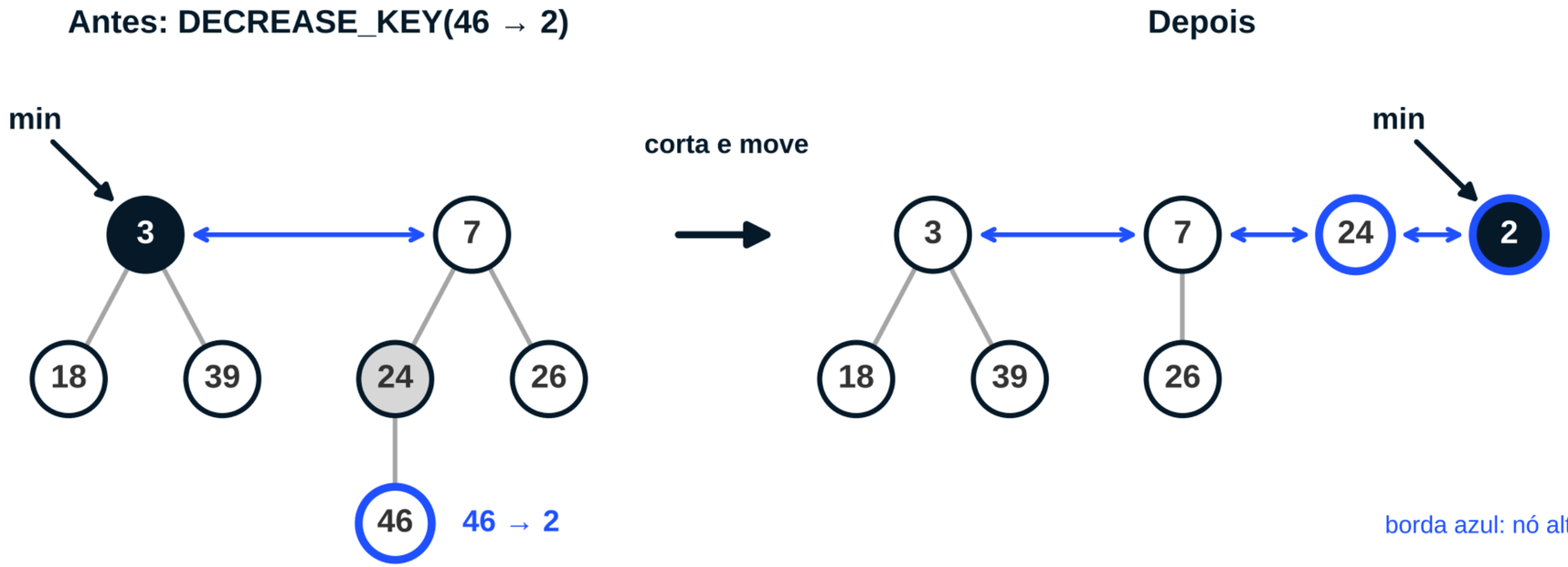


Figura 5 – Fluxograma de controle de Diminuir-Chave integrado à propagação recursiva de Corte-Em-Cascata. Elaborado pelos autores com base na implementação do grupo.

Decrease-Key

Decrease-Key



1) 2 < 24 (pai): o nó 2 é cortado e vai para a lista de raízes; como 2 < 3, o min passa a ser 2.
2) 24 já estava marcado: corte em cascata, e 24 também vai para a lista de raízes (sem marca).
3) 7 é raiz: a propagação para.

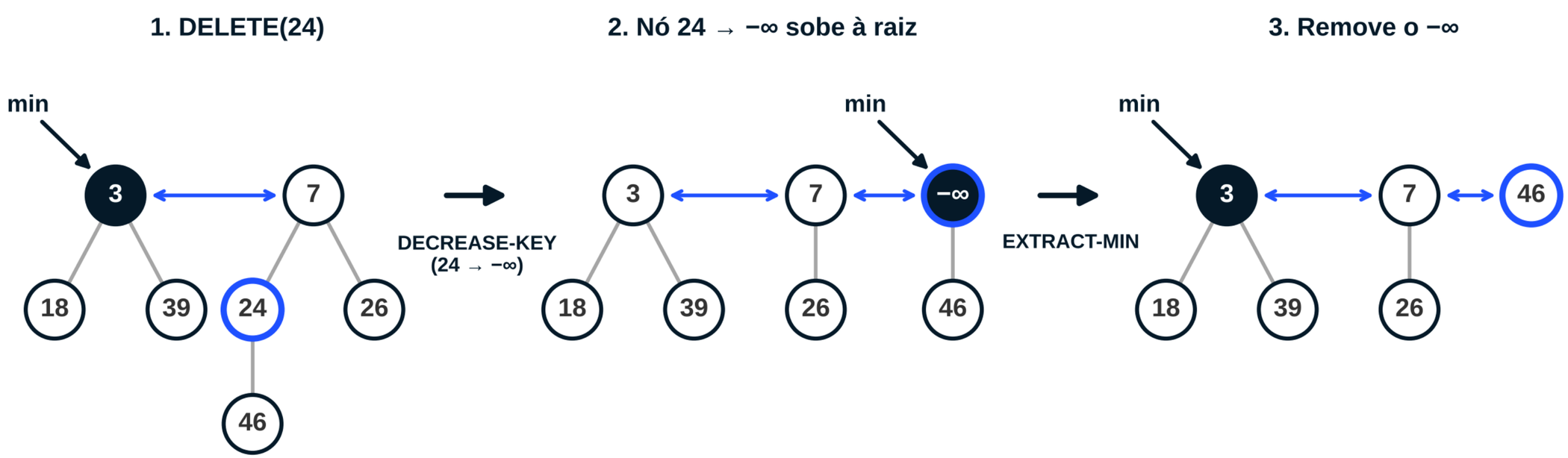
Pseudocódigos do Heap de Fibonacci

Delete

DELETE(H, x)
Saída: o nó removido
1 DECREASE-KEY$(H, x, -\infty)$;
2 **retorna** *EXTRACT-MIN*(H);

Elaborado pelos autores com base na implementação do grupo.

Delete



Nó a remover: 24
(borda azul)

A chave −∞ viola a ordem do heap: o nó é cortado e vai para a lista de raízes, tornando-se o min

O filho 46 vira raiz. As raízes têm graus distintos (2, 1 e 0), então não há uniões; o min volta a ser 3

Custo assintótico

Complexidade das operações do heap de Fibonacci

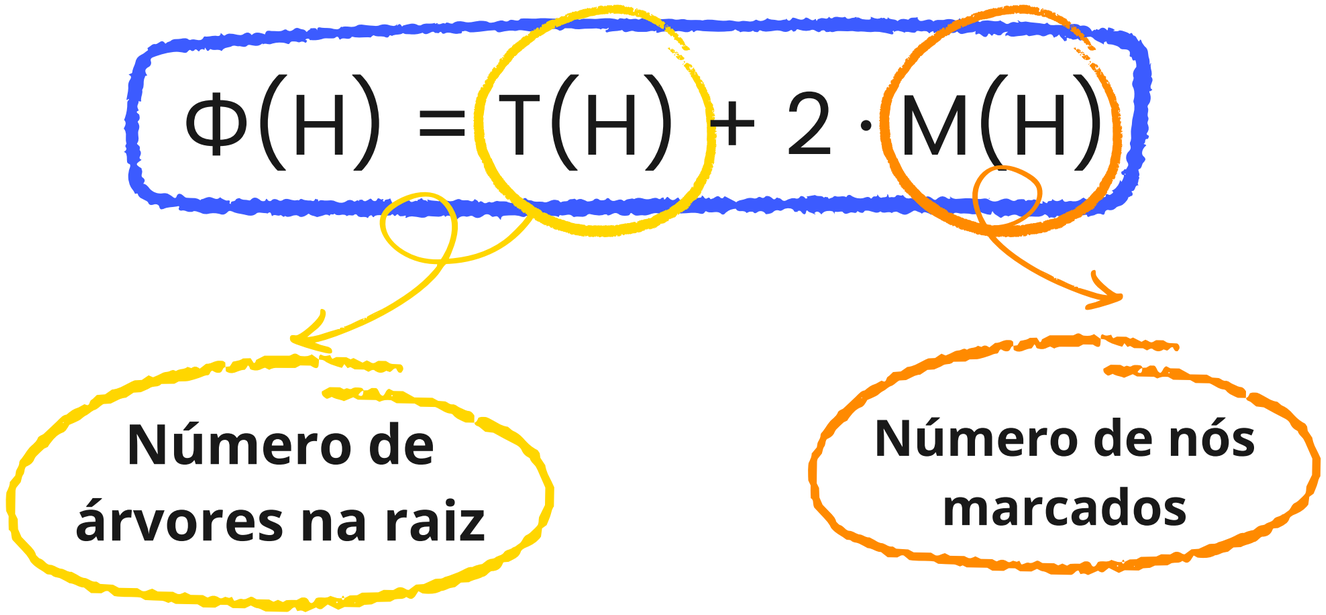
Operação	Pior caso (assintótico)	Amortizado
MAKE-HEAP	$O(1)$	$O(1)$
INSERT	$O(1)$	$O(1)$
MINIMUM	$O(1)$	$O(1)$
UNION	$O(1)$	$O(1)$
EXTRACT-MIN	$O(n)$	$O(\log n)$
DECREASE-KEY	$O(n)$	$O(1)$
DELETE	$O(n)$	$O(\log n)$

Fonte: Adaptado de Cormen et al. (2009, cap. 19, seção 19.3).

- Estratégia Lazy (Preguiçosa): As operações de **inserção, mínimo, união e redução de chave** adiam a reorganização da estrutura para o momento da extração do mínimo. Elas apenas adicionam nós à lista circular de raízes.

O método do potencial (análise amortizada)

- É uma ferramenta matemática que mede o "trabalho acumulado ou pendente" que o heap precisará resolver no futuro.



- Crédito (Aumento de Φ): Operações rápidas (como INSERT ou CUT) aumentam levemente o potencial, acumulando "crédito".
- Débito (Queda de Φ): Operações pesadas (como a consolidação no EXTRACT-MIN ou cortes em cascata) consomem esse trabalho acumulado para reorganizar a estrutura sem exceder o custo amortizado.

Aplicação real

Algoritmo de Dijkstra

Por que o Algoritmo de Dijkstra?

- Benchmark de Mundo Real: testa o Heap de Fibonacci em um cenário prático e amplamente utilizado (redes, rotas e navegação por GPS).

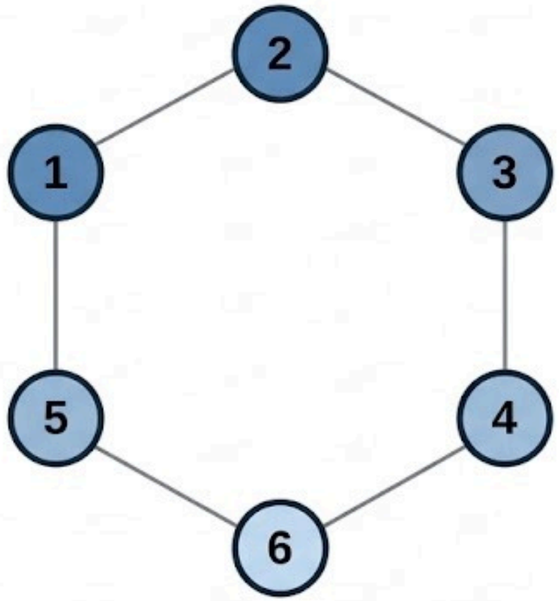
- Uso Intensivo de Operações: a execução do Dijkstra depende diretamente de duas operações centrais do heap: **MINIMUM, EXTRACT-MIN e DECREASE-KEY** (especificamente para a redução da distância dos vértices vizinhos durante o relaxamento das arestas)

Aplicação real

Algoritmo de Dijkstra

Grafo Esparso

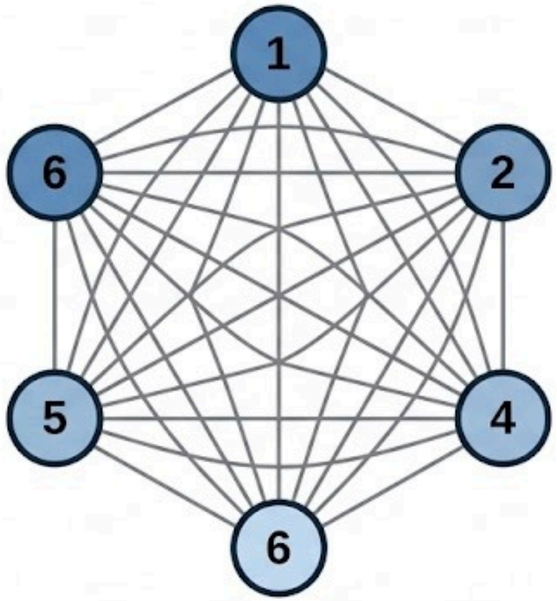
Poucas arestas em relação ao número de vértices ($|E| \approx |V|$)



Ideal para listas de adjacência.

Grafo Denso

Muitas arestas, próximo ao máximo possível ($|E| \approx |V|^2$)



Ideal para matrizes de adjacência.

Custo assintótico

Algoritmo de Dijkstra e Filas de Prioridade

Estrutura	Insert	Extract-Min	Decrease-Key	Dijkstra Total
Binário ($d = 2$)	$O(\log V)$	$O(\log V)$	$O(\log V)$	$O((V + E) \log V)$
4-ário ($d = 4$)	$O(\log_4 V)$	$O(4 \log_4 V)$	$O(\log_4 V)$	$O((V + E) \log_4 V)$
8-ário ($d = 8$)	$O(\log_8 V)$	$O(8 \log_8 V)$	$O(\log_8 V)$	$O((V + E) \log_8 V)$
Fibonacci	$O(1)$	$O(\log V)^*$	$O(1)^*$	$O(E + V \log V)^*$

$*$Indica complexidade amortizada.

Resultados

Algoritmo de Dijkstra e Filas de Prioridade

Tabela 14 – Desempenho de Dijkstra com diferentes filas de prioridade (`seed=42`).

Grafo	Fila	$\|V\|$	$\|E\|$	Tempo (ms)	Memória (KB)
Esparso	Binário ($d = 2$)	100.000	300.000	$50{,}85 \pm 8{,}01$	14.880
Esparso	4-ário ($d = 4$)	100.000	300.000	$56{,}77 \pm 10{,}43$	14.940
Esparso	8-ário ($d = 8$)	100.000	300.000	$65{,}55 \pm 8{,}43$	14.940
Esparso	Fibonacci	100.000	300.000	$471{,}37 \pm 131{,}43$	15.160
Denso	Binário ($d = 2$)	5.000	4.999.000	$68{,}93 \pm 4{,}75$	59.236
Denso	4-ário ($d = 4$)	5.000	4.999.000	$73{,}83 \pm 9{,}59$	59.236
Denso	8-ário ($d = 8$)	5.000	4.999.000	$88{,}53 \pm 33{,}95$	59.236
Denso	Fibonacci	5.000	4.999.000	$103{,}09 \pm 36{,}98$	59.488

Resultados

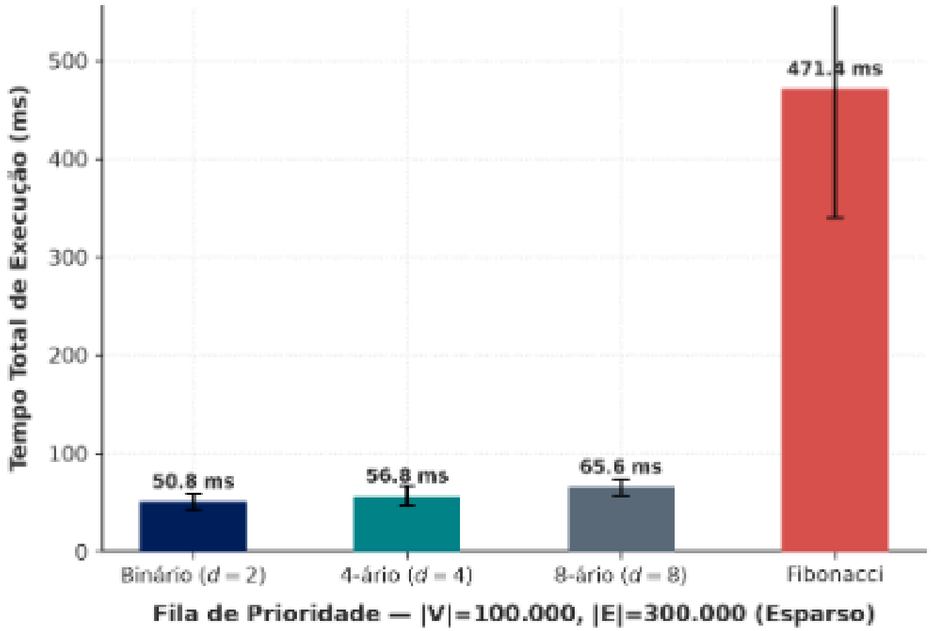
- Heap Binário (d=2): vencedor na prática em todos os cenários
  - Na teoria possui custo logarítmico em todas as operações (O(log V)).
  - Na prática: estrutura simples (vetor contíguo), sem overhead de ponteiros e com excelente localidade de referência para a memória cache.

- Heaps d-ários (d=4, 8): impacto da Aridade
  - Na teoria as árvores são mais baixas (log_d V), acelerando o DECREASE-KEY em grafos densos.
  - Na prática: ficaram ligeiramente atrás do binário (d=2 < d=4 < d=8).

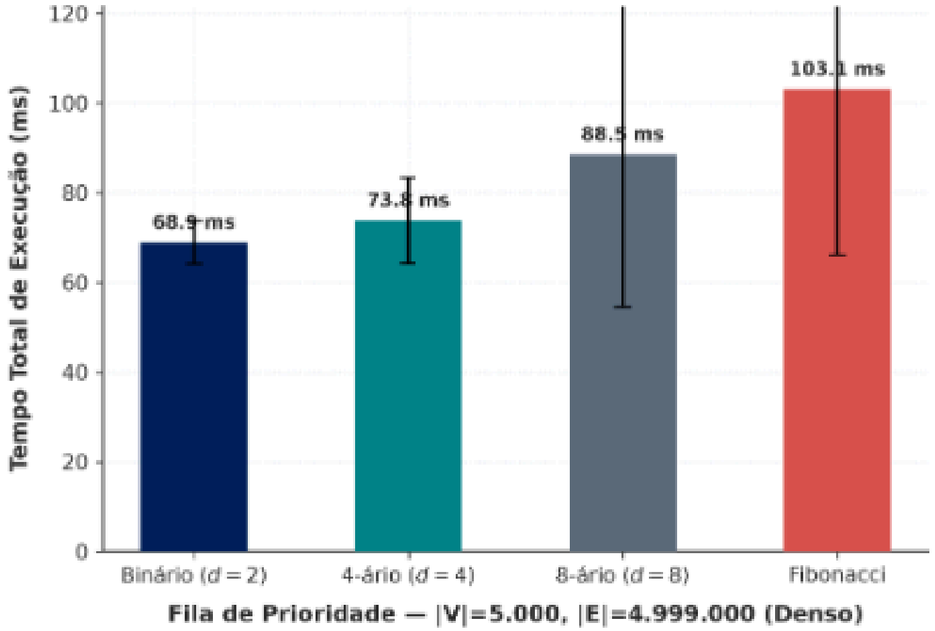
- Heap de Fibonacci: discrepância Teoria vs. Prática
  - Na teoria possui O(1) amortizado para o DECREASE-KEY prometia a melhor complexidade assintótica no Dijkstra (O(E + V log V)).
  - Prática no Esparso: foi cerca de 9x mais lento que o binário (471 ms vs. 50 ms) Prática no Denso: reduziu a diferença para 1.5x mais lento (103 ms vs. 68 ms).
  - Motivo: A alta constante oculta (alocação dinâmica de memória, manutenção de listas duplamente encadeadas e marcações de nós) sobressaiu sobre o ganho assintótico teórico.

Resultados

Figura 31 – Tempo total médio de execução de Dijkstra, em ms, com diferentes filas de prioridade; as barras de erro representam um desvio padrão das dez execuções. Os painéis identificam o tamanho e a densidade de cada grafo.



(a) Grafo esparso: $|V| = 100.000$ e $|E| = 300.000$

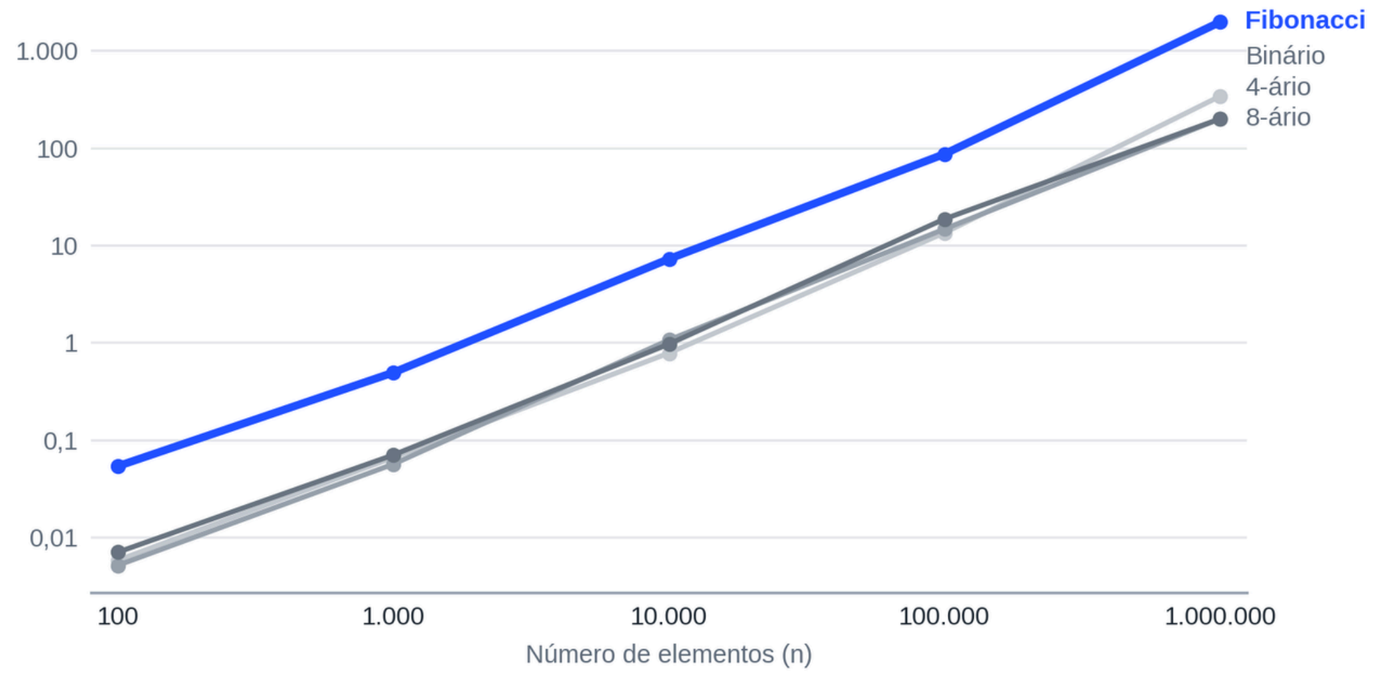


(b) Grafo denso: $|V| = 5.000$ e $|E| = 4.999.000$

Resultados

Na extração do mínimo, o Fibonacci é 5,8× mais lento que o binário com n = 1.000.000, apesar do mesmo custo O(log n)

Tempo mediano para esvaziar o heap (entrada aleatória), em ms; escala logarítmica nos dois eixos

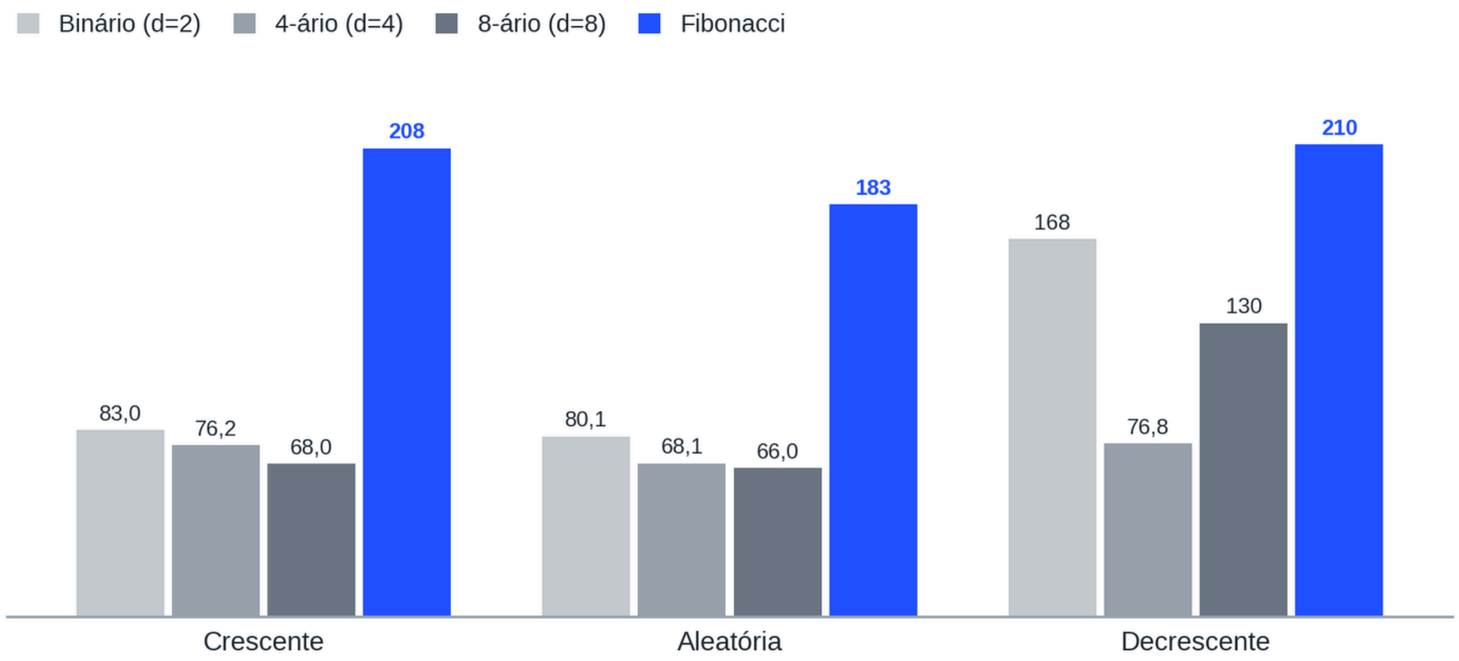


Fonte: experimentos do grupo (repositório AEDS-AdvancedStructures), implementação com std::vector

Resultados

Em reduções de chave com $n = 10^6$, o Fibonacci levou de 2,7× a 3,1× o tempo do melhor d-ário, nas três ordens de entrada

Tempo mediano de 10^6 reduções de chave ($n = 10^6$), em ms



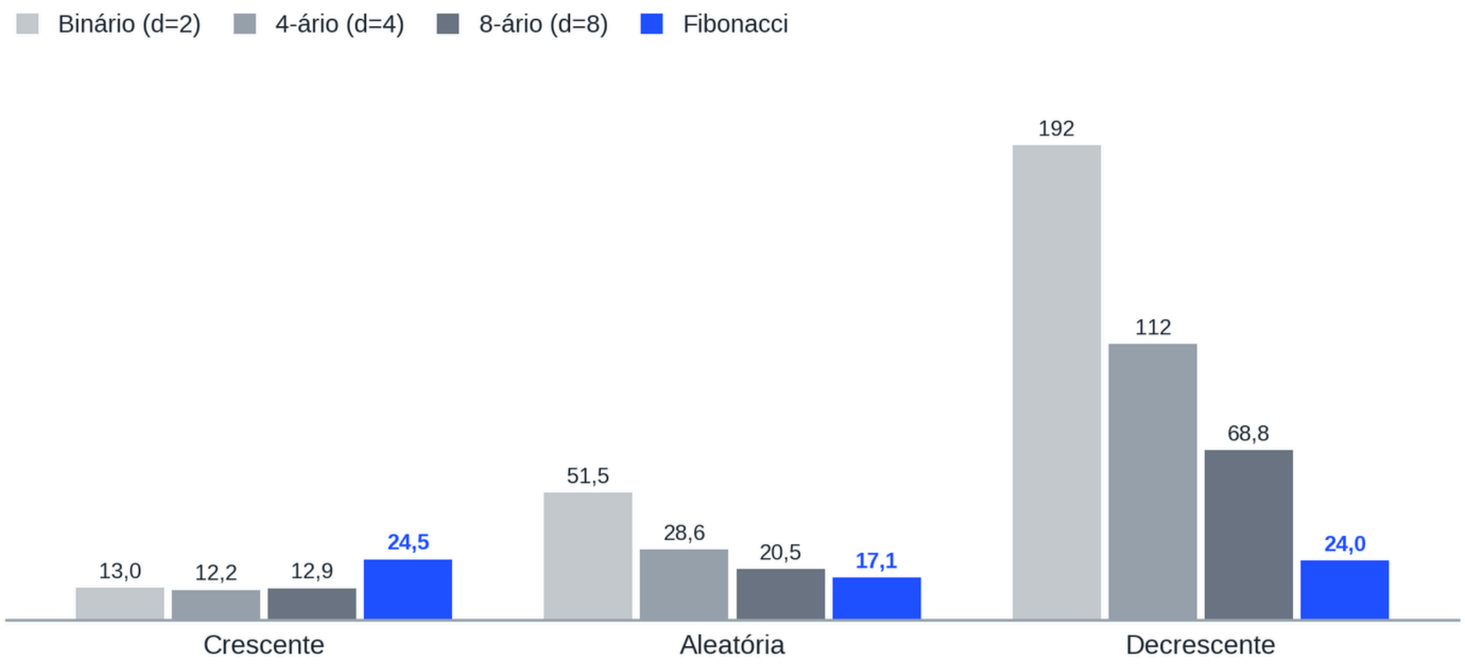
Nota: ordem da entrada = ordem das chaves inseridas antes das reduções.

Fonte: experimentos do grupo (repositório AEDS-AdvancedStructures), implementação com std::vector

Resultados

A inserção do Fibonacci varia pouco com a ordem da entrada (17,1 a 24,5 ms); a do binário varia de 13,0 a 192 ms

Tempo mediano para inserir 10^6 elementos ($n = 10^6$), em ms



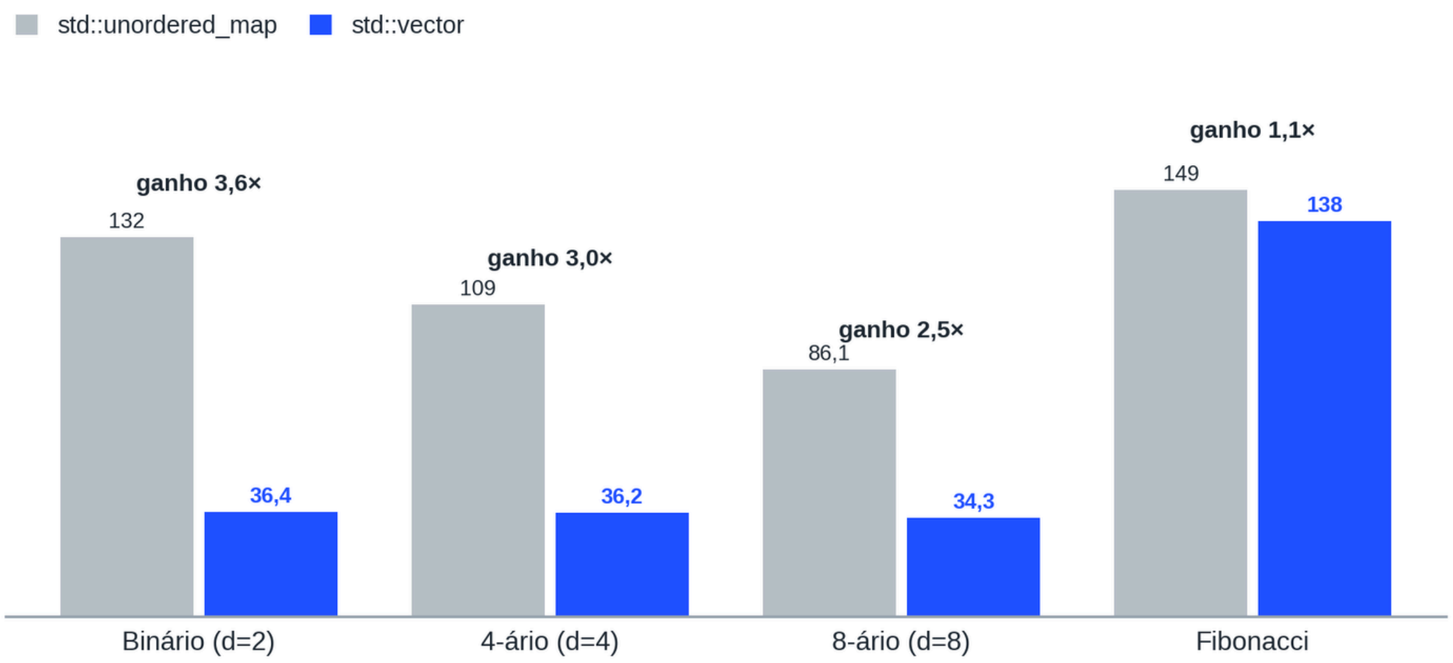
Nota: a destruição do heap não entra na medição.

Fonte: experimentos do grupo (repositório AEDS-AdvancedStructures), implementação com std::vector

Resultados

Trocar unordered_map por vector acelerou os heaps d-ários em 2,5× a 3,6× e o de Fibonacci em apenas 1,1×

Tempo mediano do Dijkstra no grafo esparso (V = 100.000; E = 300.000), em ms



Nota: o ganho indicado acima de cada grupo é o tempo com unordered_map dividido pelo tempo com vector.

Fonte: experimentos do grupo (repositório AEDS-AdvancedStructures), duas versões da tabela de posições

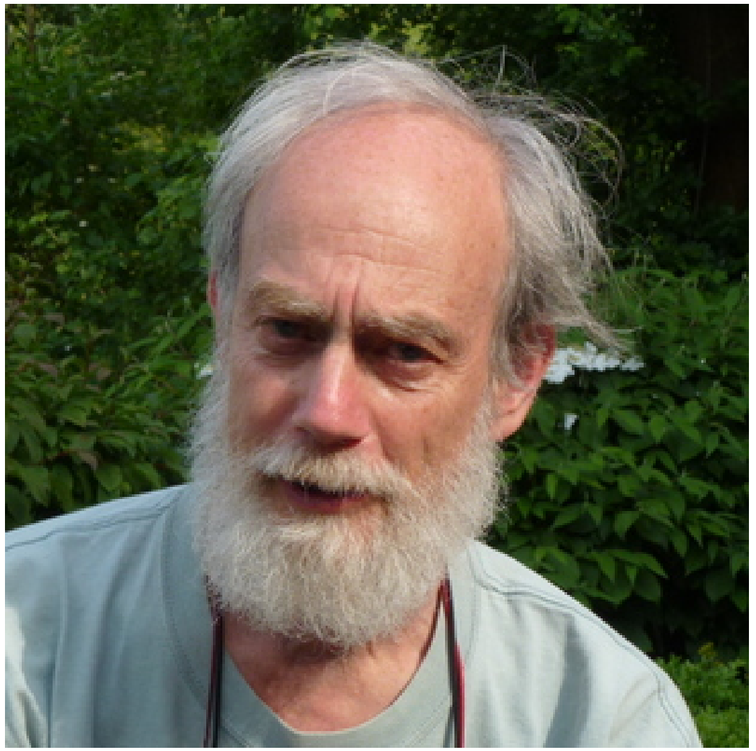
Árvore de van Emde Boas

Decomposição recursiva do universo de chaves e operações em tempo O(log log U)

O Criador & a Motivação:

O autor: **Peter van Emde Boas** é matemático, cientista da Computação e professor universitário em Amsterdã.

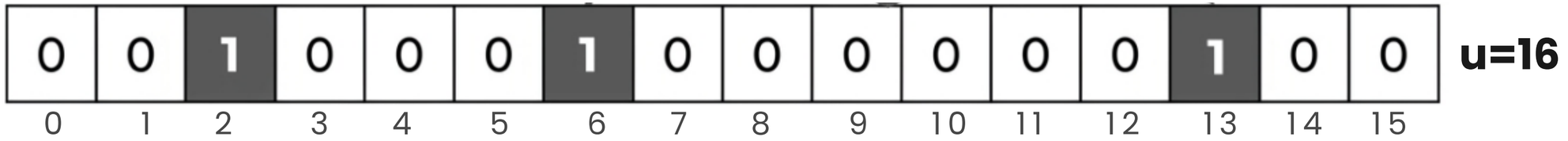
Motivação: **Superar o limite das comparações**.
Ele queria saber se era possível realizar operações de conjunto dinâmico e fila de prioridade (Insert, Delete, Member, Successor, Predecessor) em tempo sub-logarítmico ou seja, **menor que O(log n)**.



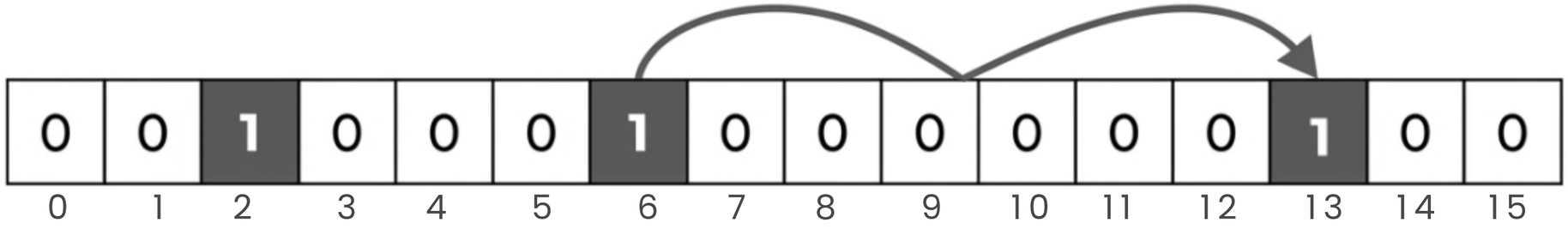
PETER VAN EMDE BOAS

Passo 1: Bit Vector

Solução: Uma estrutura que armazena 1 se o elemento estiver presente e 0 se não estiver presente no universo u.



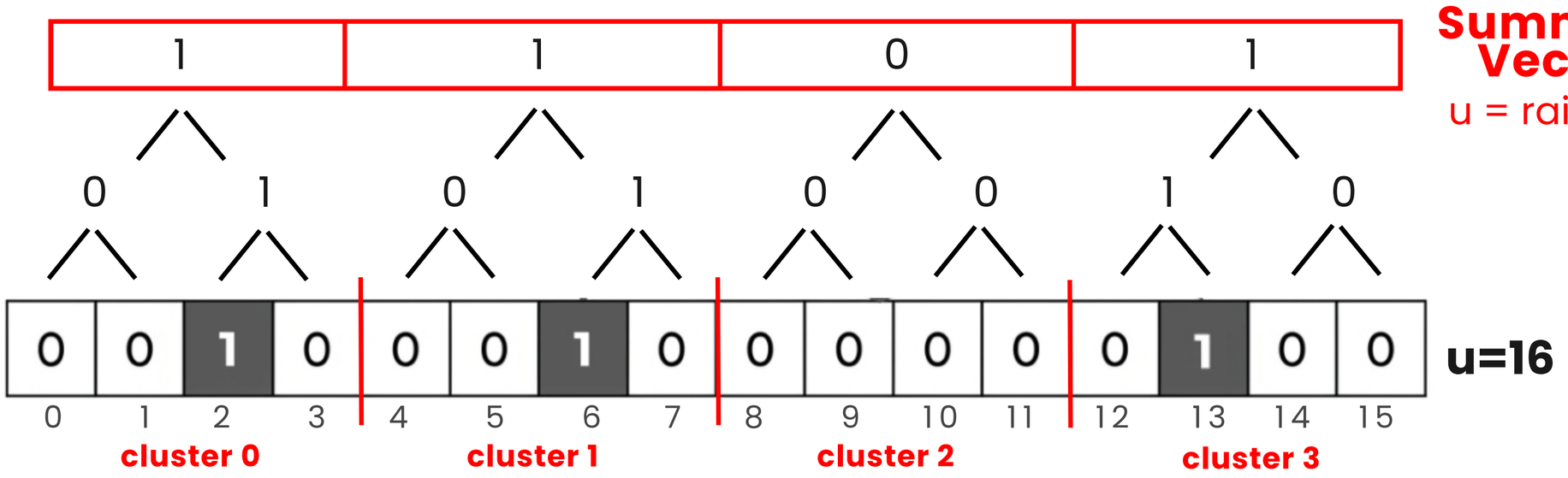
Problema: Sucessor custa O(u) no pior caso.



Passo 2: Clusterização & Summary Vector

Solução:

- Dividimos o vetor em blocos de tamanho raiz de u chamados **cluters**
- Criamos um vetor auxiliar chamado ‘Summary’ que indica se há ou não elementos nas proximas posições.

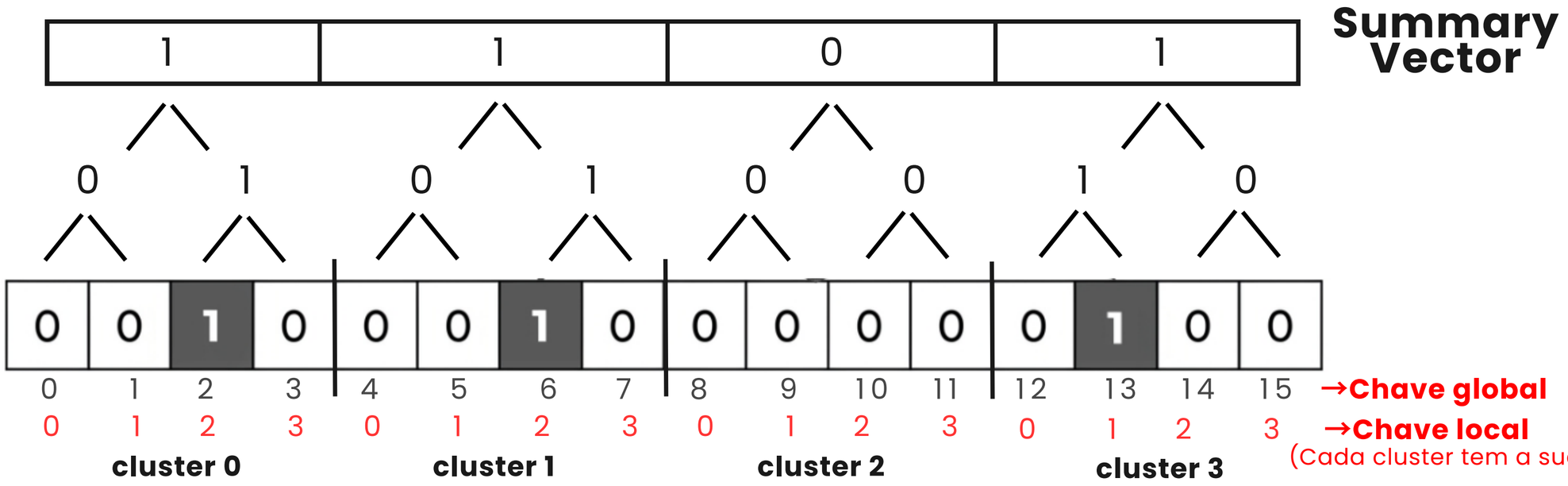


Passo 2: Clusterização & Summary Vector

Sucessor(6):

1. Procuramos no cluster do 6 → Não há sucessor.
2. Procuramos pelo próximo '1' no Summary Vector e "descemos".
3. Procuramos pelo primeiro elemento naquele cluster.

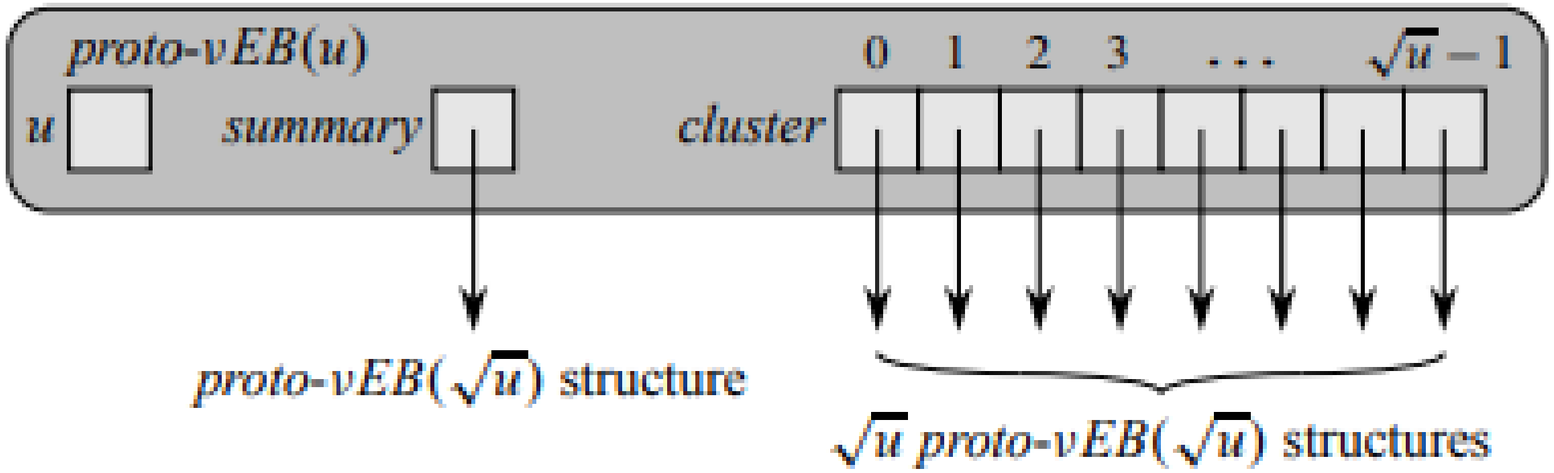
O(raiz(u))



Passo 3: Estrutura Proto-vEB

Solução:

- Aplica a ideia de divisão por raiz de u **recursivamente** tanto para os clusters quanto para o summary, até atingir o caso base **u = 2.**
- Cada Cluster e summary tornam-se estruturas Proto-vEB independentes.

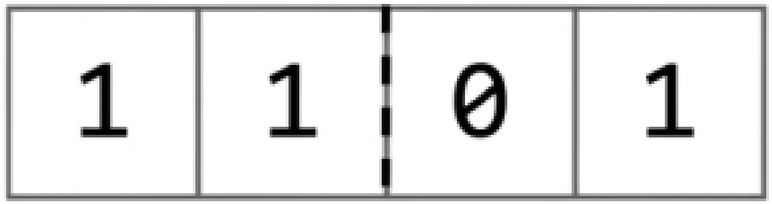


Passo 3: Estrutura Proto-vEB

Coordenadas de Bits: Cada chave x é fatiada em dois blocos de bits

Exemplo: x = 13 (decimal)

13 (binário)



$$\text{high}(x) = \lfloor x / \sqrt{u} \rfloor$$

13 / 4 = **3**

$$\text{low}(x) = x \bmod \sqrt{u}$$

13 mod 4 = **1**

Metade Alta (high)

Metade Baixa (low)

11(b) = **3**(d)

01(b) = **1**(d)

Cluster

Posição no Clúster

Passo 3: Estrutura Proto-vEB

Coordenadas de Bits: Cada chave x é fatiada em dois blocos de bits

Exemplo: x = 13 (decimal)

13 (binário)

$$\text{high}(x) = \lfloor x / \sqrt{u} \rfloor$$

13 / 4 = **3**

1	1	0	1

$$\text{low}(x) = x \bmod \sqrt{u}$$

13 mod 4 = **1**

Metade Alta (high)

Metade Baixa (low)

11(b) = **3**(d)

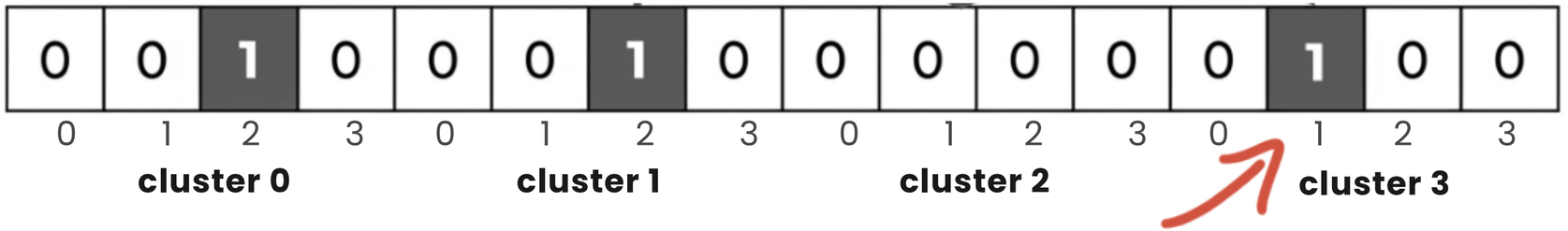
Cluster

01(b) = **1**(d)

Posição no Cluster

→ O número 13 vai ser armazenado na posição 1 do cluster 3

Passo 3: Estrutura Proto-vEB



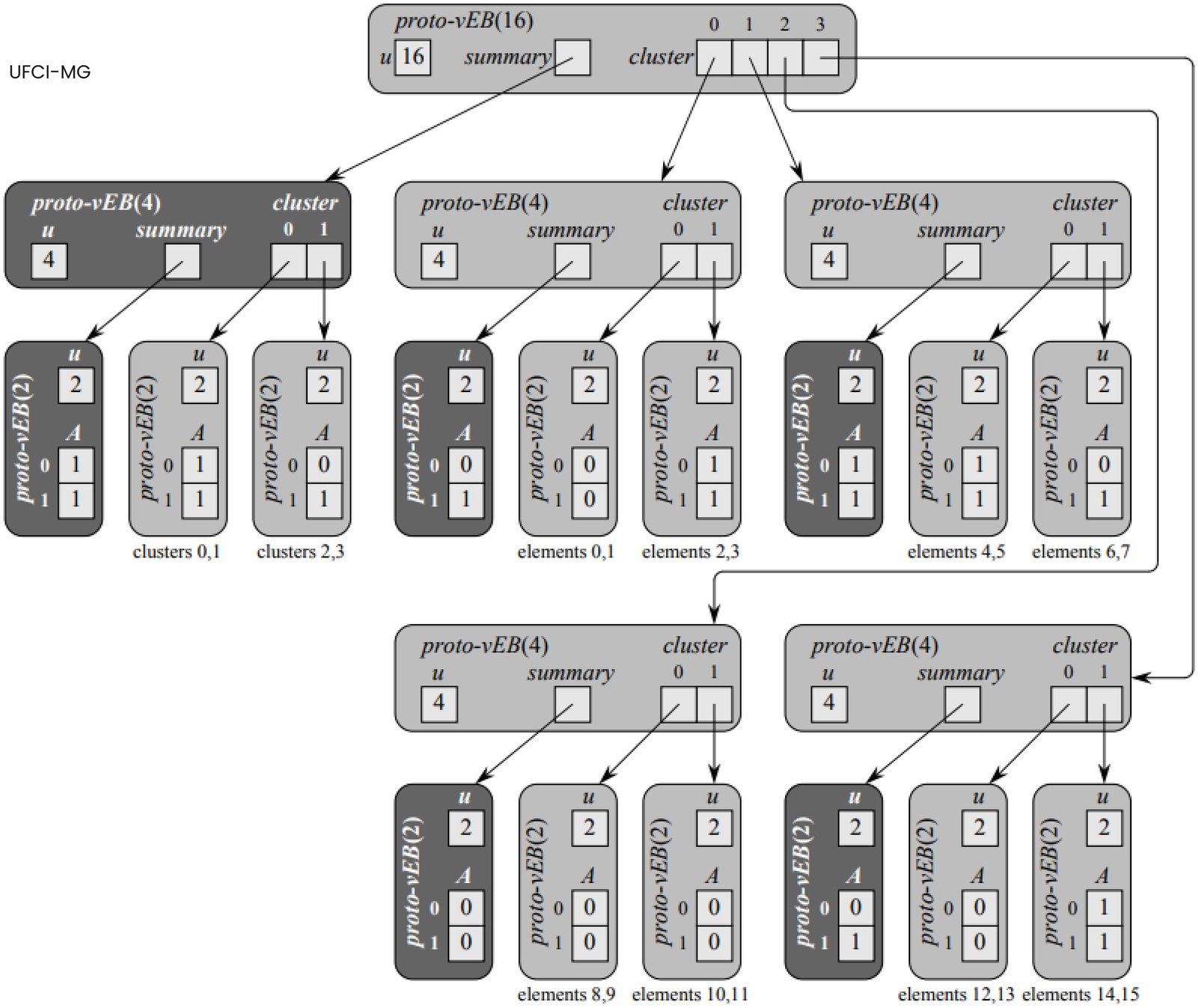
Caminho inverso:

Posição no cluster

$$\text{index}(x, y) = x\sqrt{u} + y$$

cluster

3*4+1 = 13



→ Representação de uma Proto-vEB de U=16 e com os elementos: {2, 3, 4, 5, 7, 14, 15}

Passo 3: Estrutura Proto-vEB

Gargalo: Buscar um sucessor ausente gera duas recursões. O algorítmo busca no cluster atual e também no summary.

Complexidae: O(log u).

Resultado: Empata com árvores binárias de busca comuns (como AVL e Red-Black), sem ganho assintótico.

Passo 4: Introdução dos Atributos min e max

Solução: Armazenamento direto do menor (min) e maior (max) elemento no topo de cada nó.

O "Pulo do Gato": O valor de min fica isolado no topo e não desce para os subclusters.

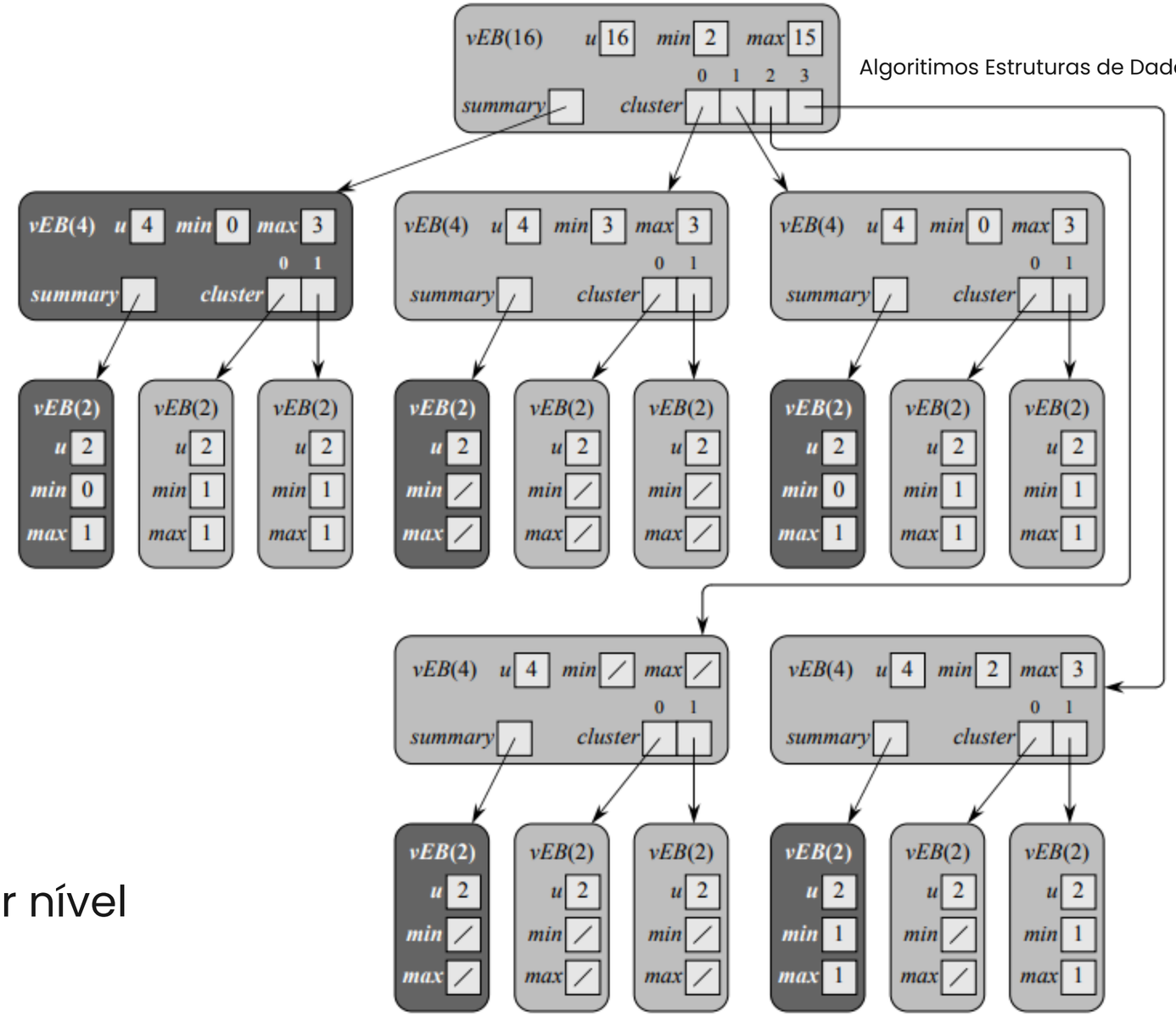
Avanço: Consultas Minimum e Maximum resolvidas em O(1) permite saber em tempo constante se uma subárvore está vazia.

Impacto: Prepara a estrutura para encurtar o caminho das chamadas recursivas.

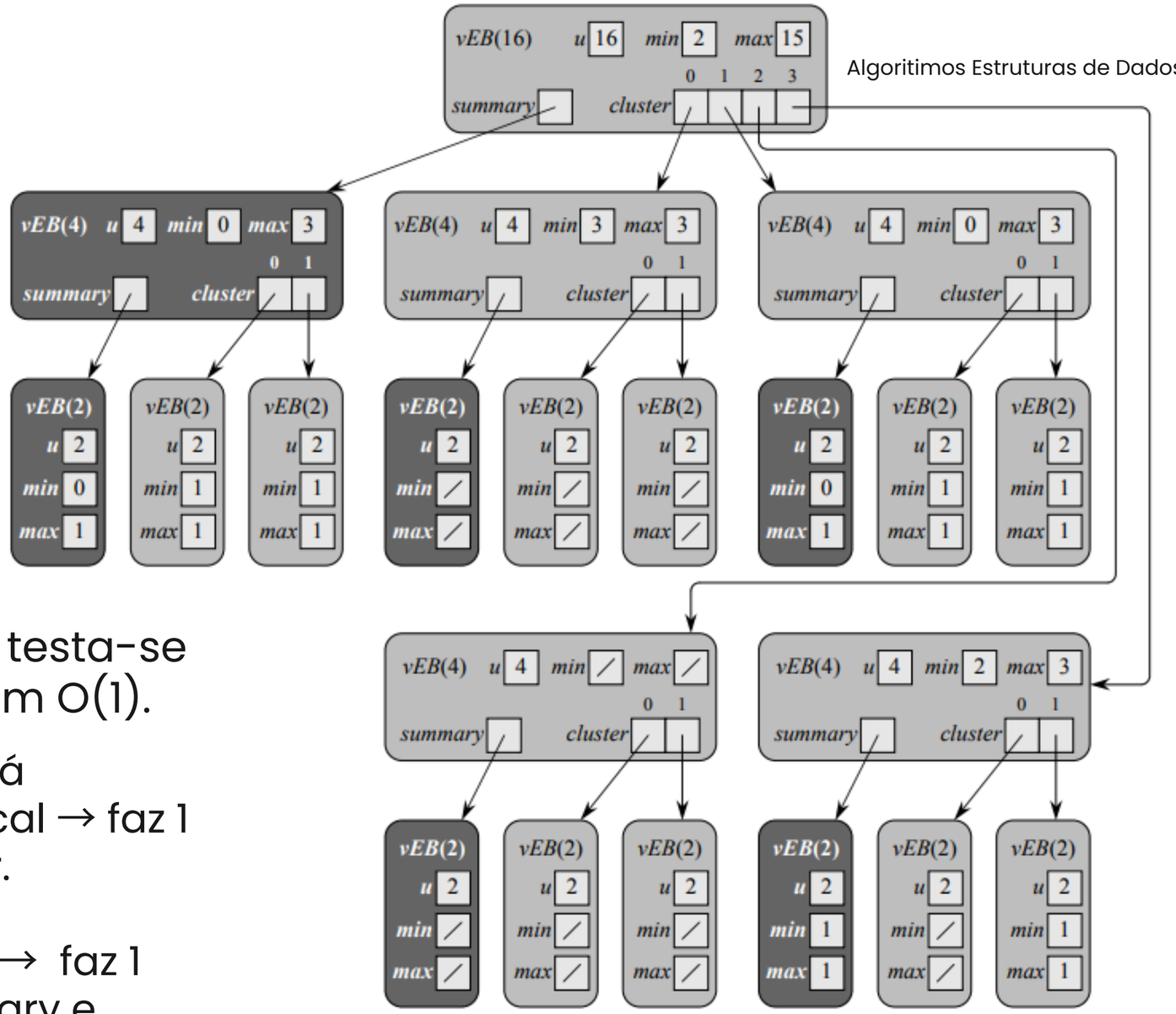
Passo 5: Van Amde Boas Tree

Solução:

- Uso de min e max para realizar testes de fronteira em O(1) antes de decidir onde recursar.
- Garante no máximo 1 chamada recursiva por nível de árvore



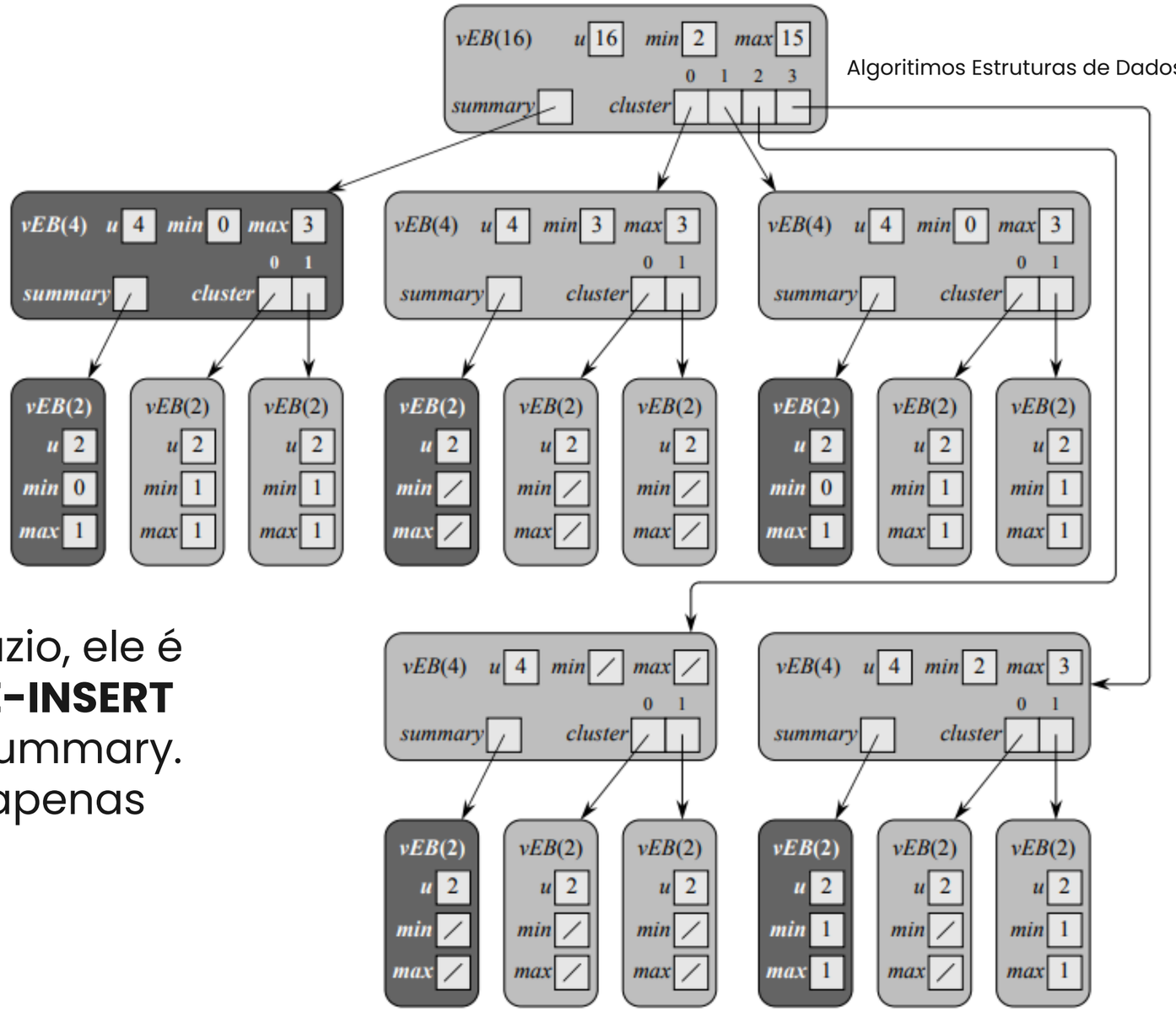
Passo 5: Van Amde Boas Tree



No Sucessor(V, x):

- Antes de descer no cluster local, testa-se **low(x) < cluster[high(x)].max** em O(1).
  - **Se verdadeiro:** O sucessor está garantidamente no cluster local → faz 1 chamada recursiva no cluster.
  - **Se falso**: O sucessor está fora → faz 1 chamada recursiva no summary e obtém o menor valor do novo cluster em O(1) lendo **cluster.min**.

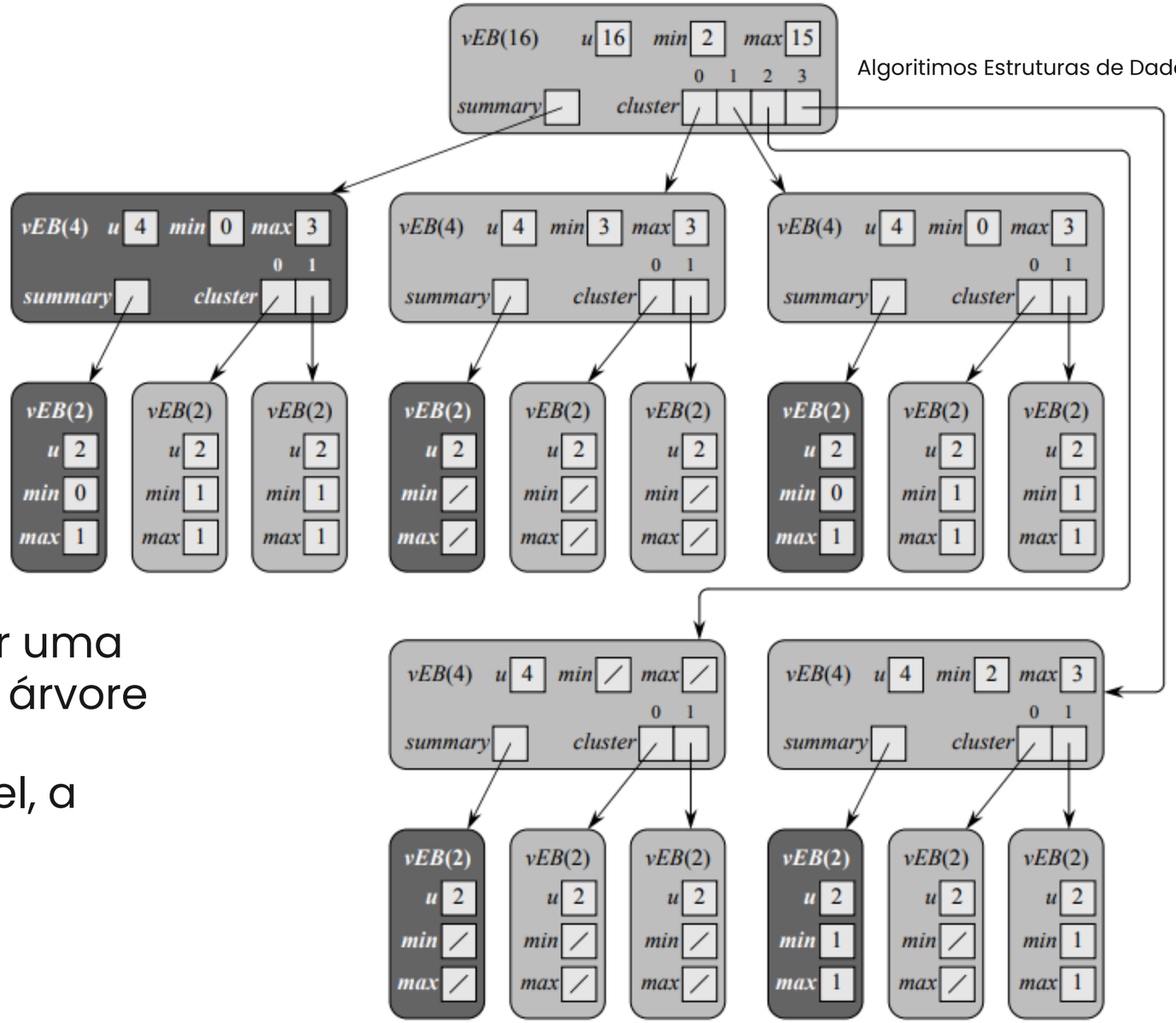
Passo 5: Van Amde Boas Tree



Na Insert(V, x):

- Se o cluster de destino estiver vazio, ele é ativado em O(1) via **EMPTY-TREE-INSERT** e a recursão desce apenas no summary. Se já estivesse ocupado, desce apenas no cluster.

Passo 5: Van Amde Boas Tree



Complexidade Final:

- O mecanismo equivale a realizar uma busca binária sobre os níveis da árvore
- Com apenas 1 chamada por nível, a recorrência vira:

O (log log u)

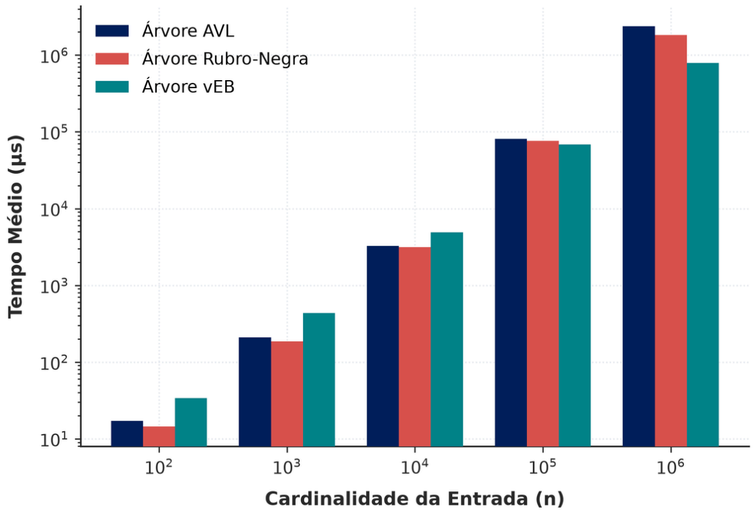
Análise assintótica

Operação	Chamadas	Custo (pior caso)
Mínimo, máximo	0	$\Theta(1)$
Pertinência	1	$O(\log \log u)$
Sucessor, predecessor	1 (cluster ou *summary*)	$O(\log \log u)$
Inserção	1 (cluster ou *summary*)	$O(\log \log u)$
Remoção	2, uma custosa	$O(\log \log u)$
Extração do mínimo	remoção $+ O(1)$	$O(\log \log u)$
Redução de chave	remoção + inserção	$O(\log \log u)$

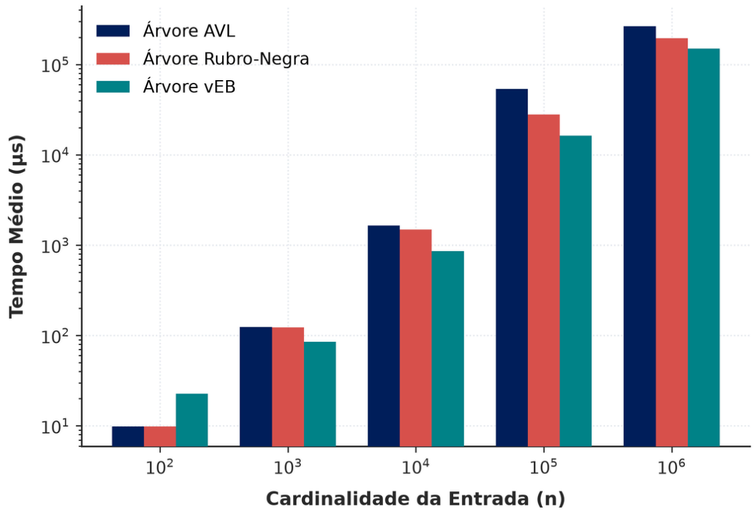
Resultados

Tempo médio de **inserção**, em µs, nas árvores **AVL**, **rubro-negra** e **vEB** em função de **n**.

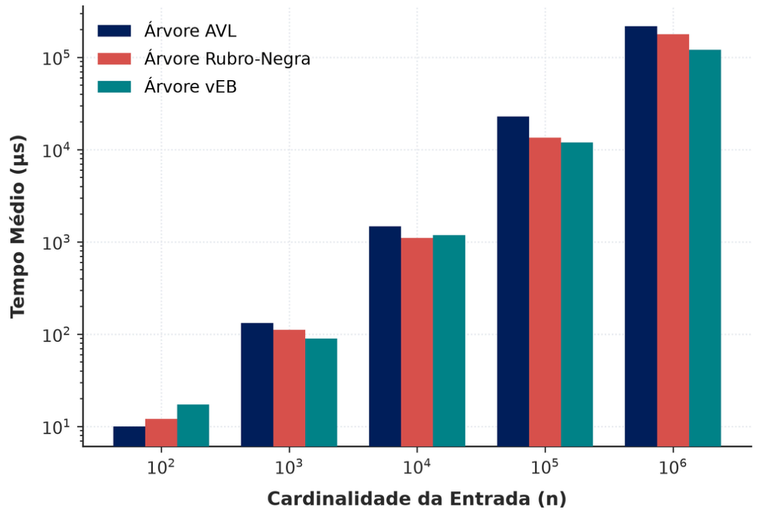
Padrão aleatório



Padrão crescente



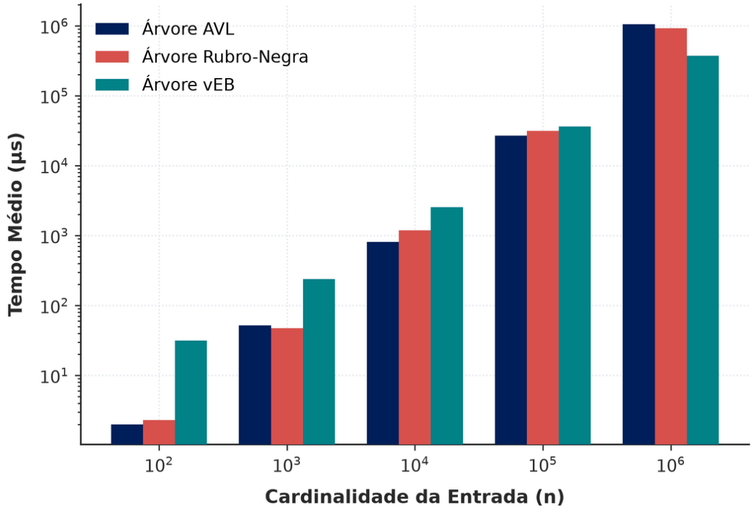
Padrão decrescente



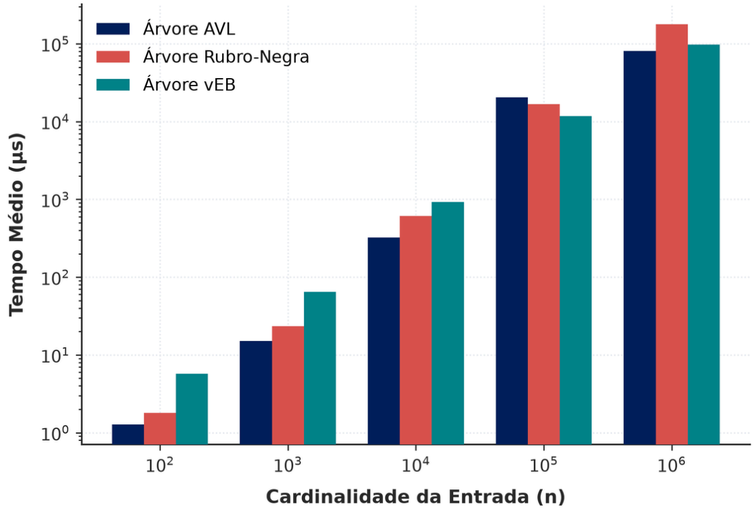
Resultados

Tempo médio de **busca por pertinência**, em µs, nas árvores **AVL**, **rubro-negra** e **vEB** em função de **n**.

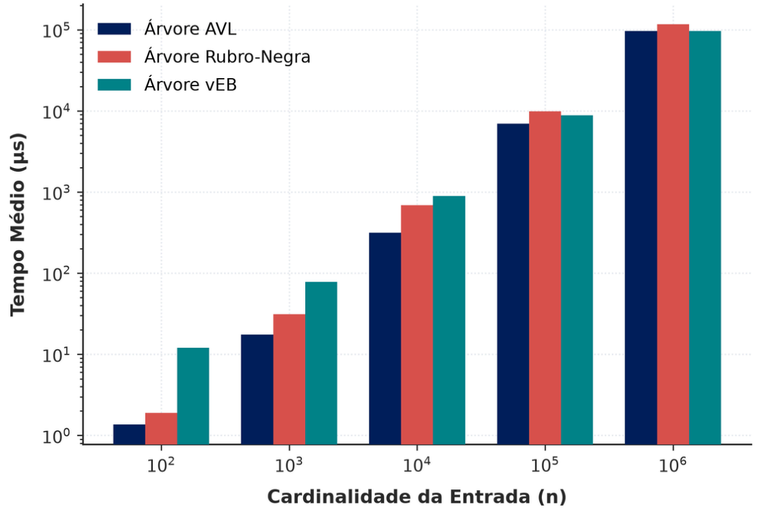
Padrão aleatório



Padrão crescente



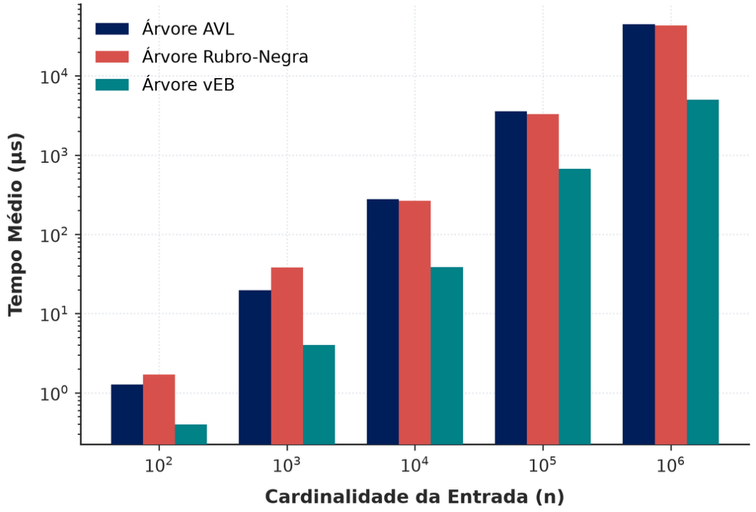
Padrão decrescente



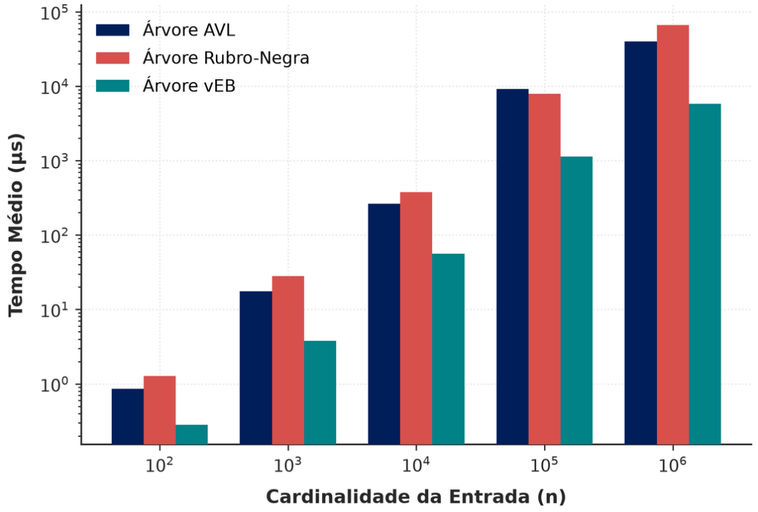
Resultados

Tempo médio de **consulta de mínimo e máximo**, em µs, nas árvores **AVL**, **rubro-negra** e **vEB** em função de **n**.

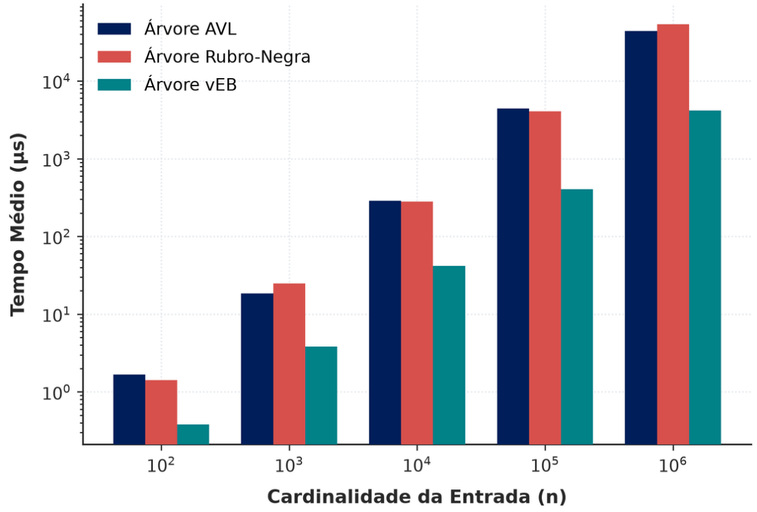
Padrão aleatório



Padrão crescente



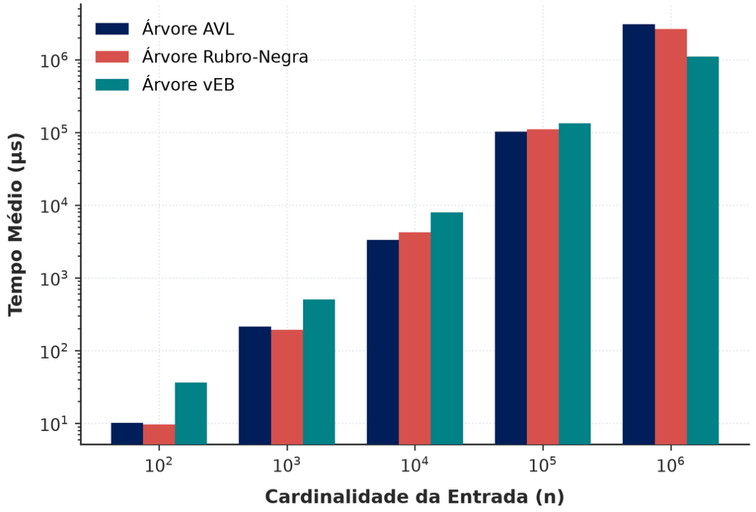
Padrão decrescente



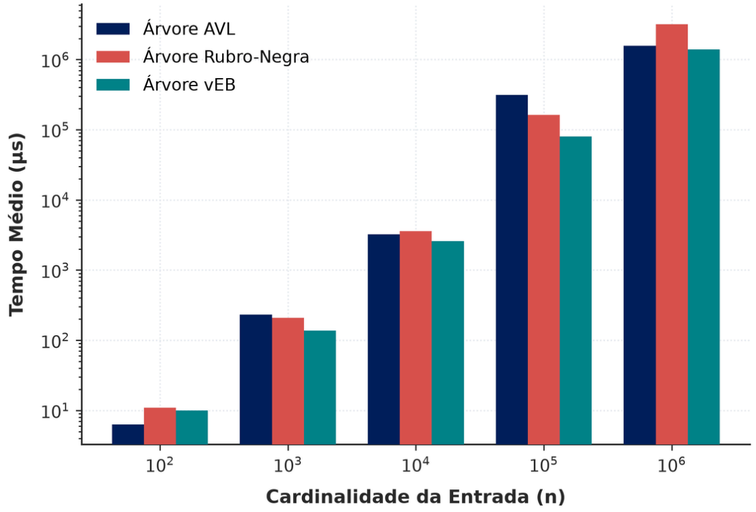
Resultados

Tempo médio de **consulta de sucessor e predecessor**, em µs, nas árvores **AVL**, **rubro-negra** e **vEB** em função de **n**.

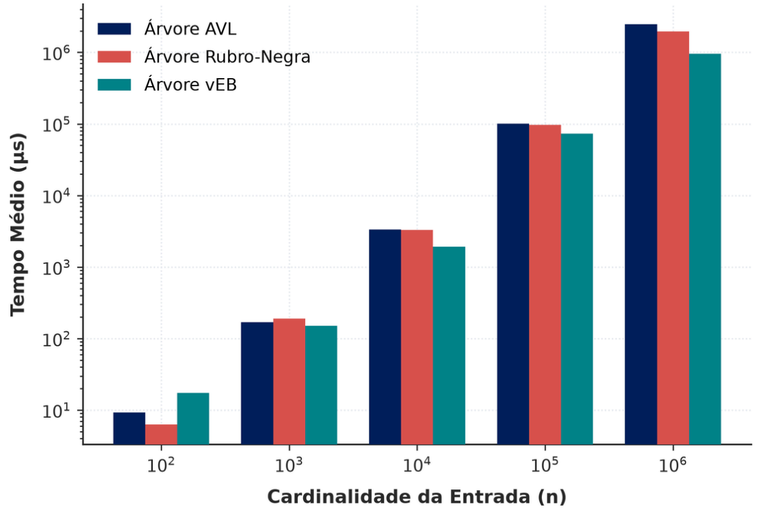
Padrão aleatório



Padrão crescente



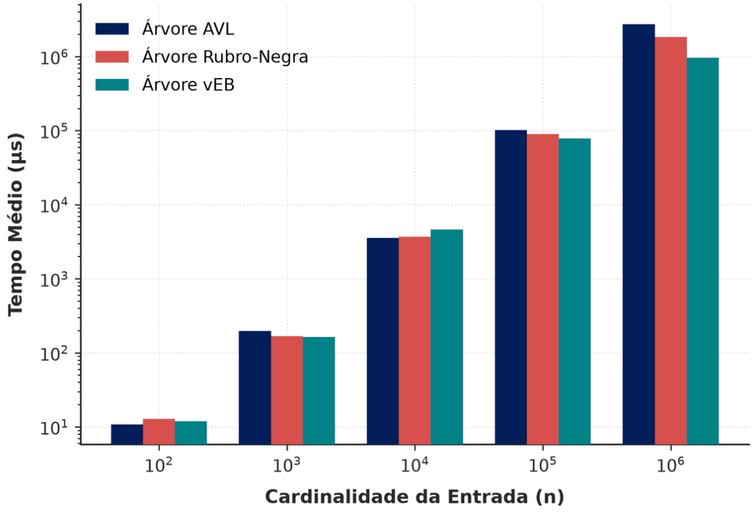
Padrão decrescente



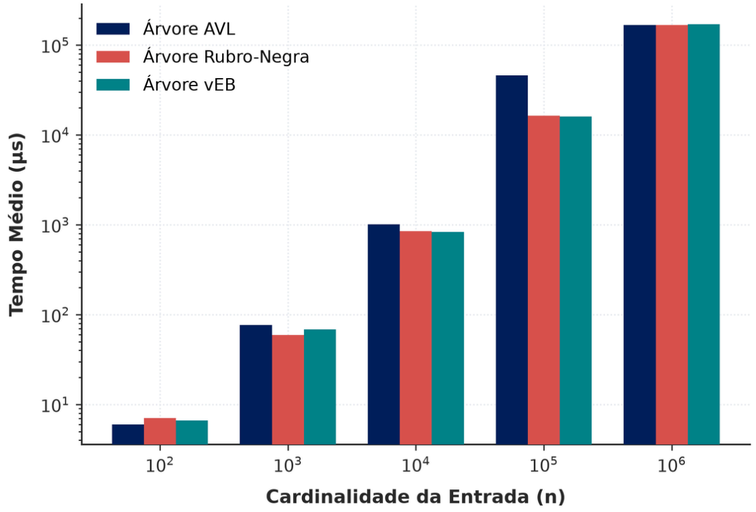
Resultados

Tempo médio de **remoção**, em µs, nas árvores **AVL**, **rubro-negra** e **vEB** em função de **n**.

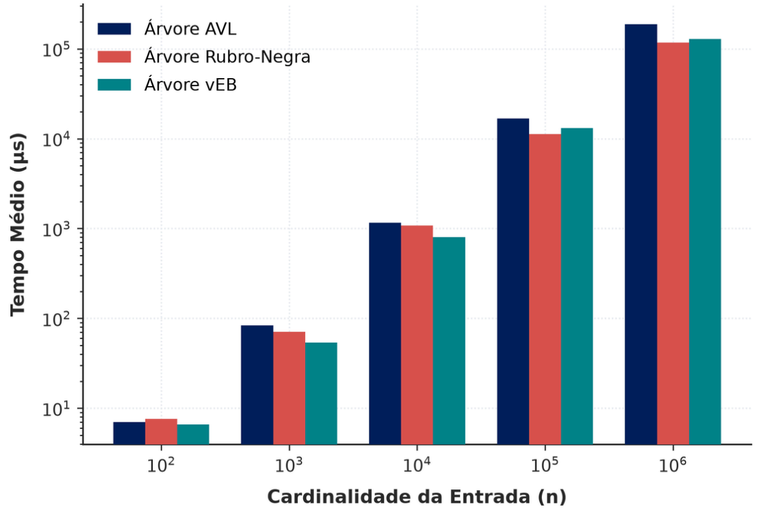
Padrão aleatório



Padrão crescente



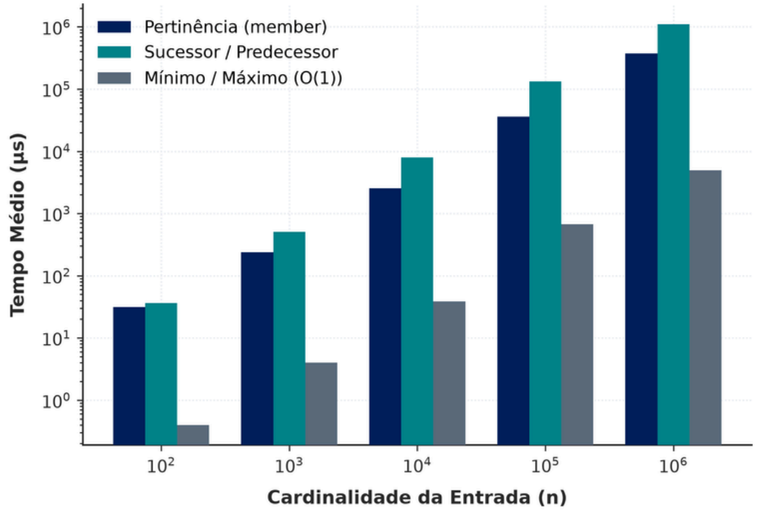
Padrão decrescente



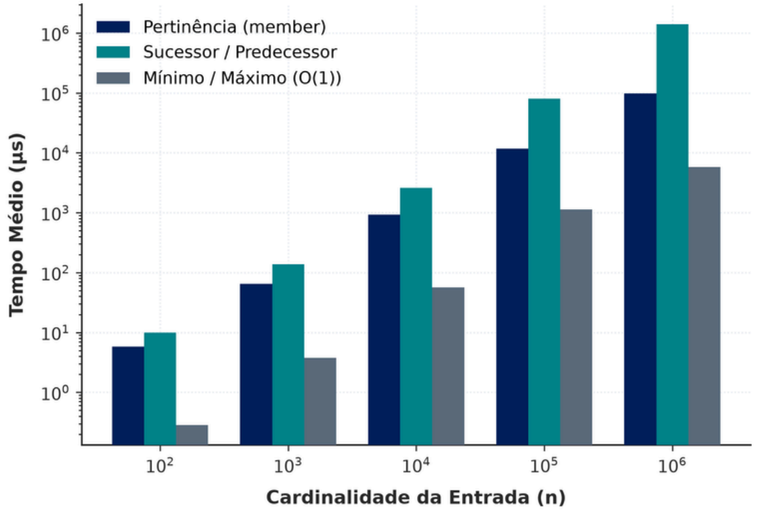
Resultados operações específicas

Tempos das operações específicas da árvore vEB, em µs: busca por **pertinência**, **sucessor/predecessor** e **consulta do máximo**, em função de **n**.

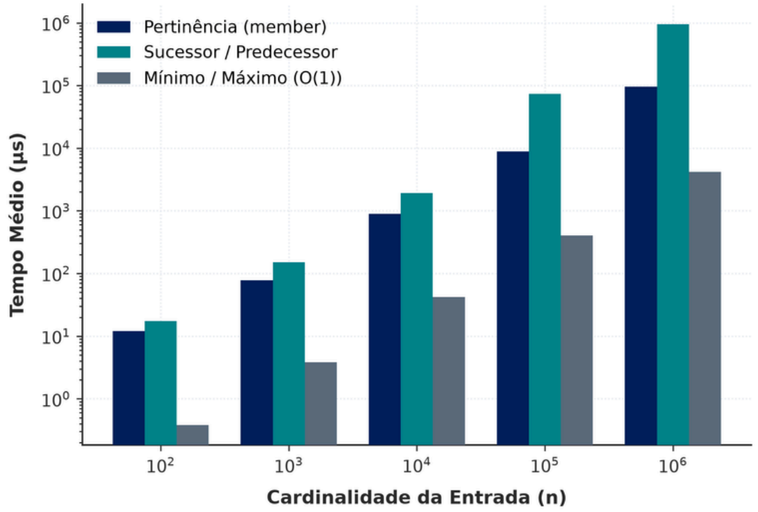
Padrão aleatório



Padrão crescente



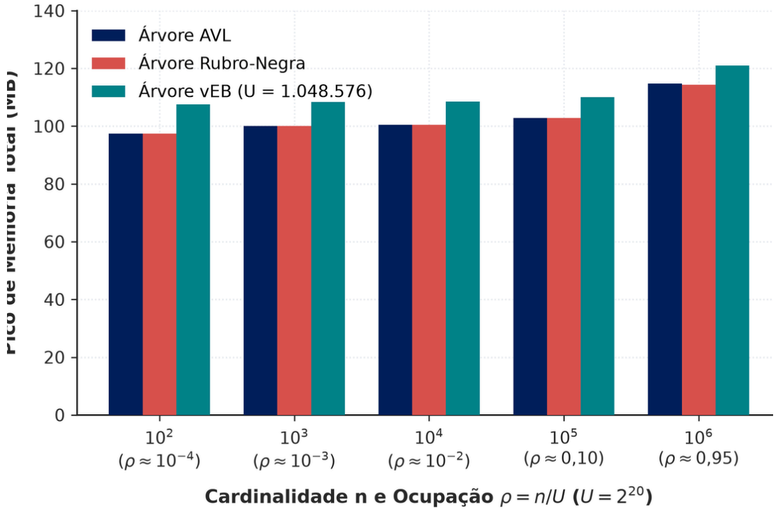
Padrão decrescente



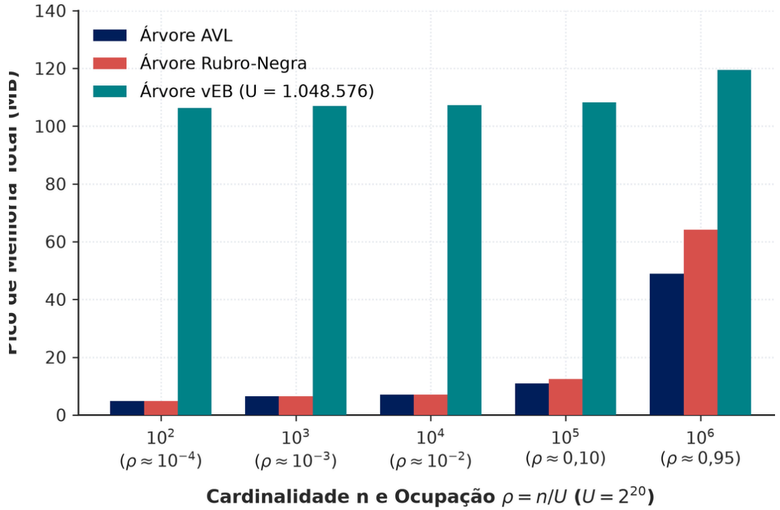
Resultados pico de memória

Pico de memória das árvores **AVL**, **rubro-negra** e **vEB** em função de **n**, com universo da vEB fixado em **U=1.048.576**.

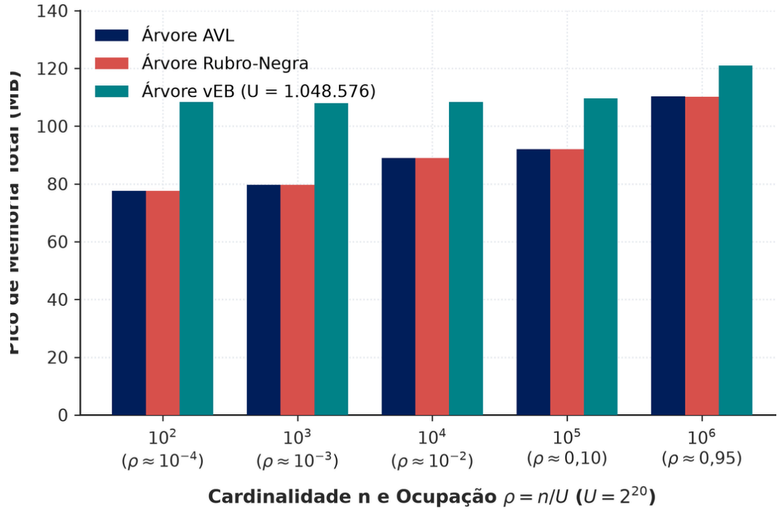
Padrão aleatório



Padrão crescente



Padrão decrescente



Conclusão

Heap de Fibonacci vs. Árvore vEB

Heap de Fibonacci

É a melhor opção quando você precisa fazer muitas **reduções de chave** (**tempo O(1) amortizado**) ou **juntar filas** de prioridade rapidamente.

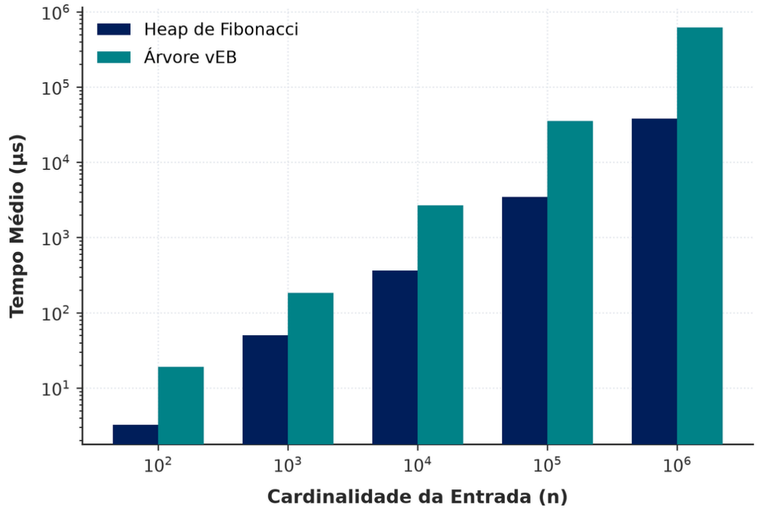
Árvore de van Emde Boas

É extremamente rápida **(O(log log U))** para encontrar o elemento **anterior ou o próximo** (predecessor e sucessor), além do **mínimo e máximo**.

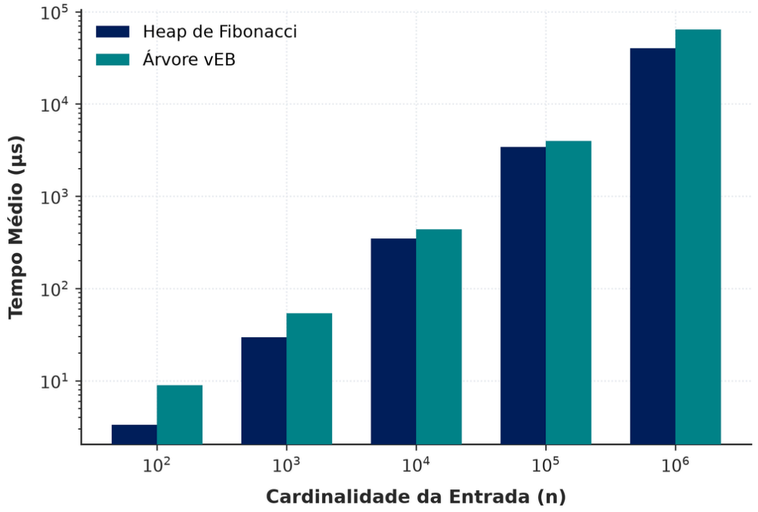
Comparação inserção

Tempo médio de **inserção**, em µs, no **heap de Fibonacci** e na **árvore vEB** em função de **n**.

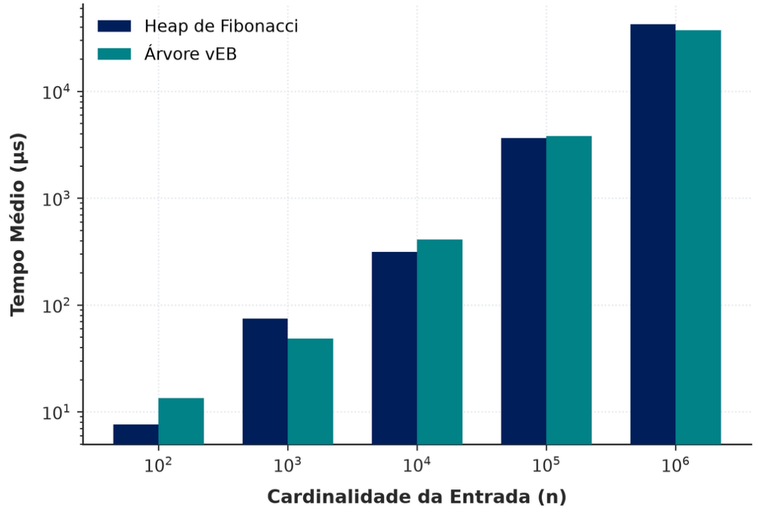
Padrão aleatório



Padrão crescente



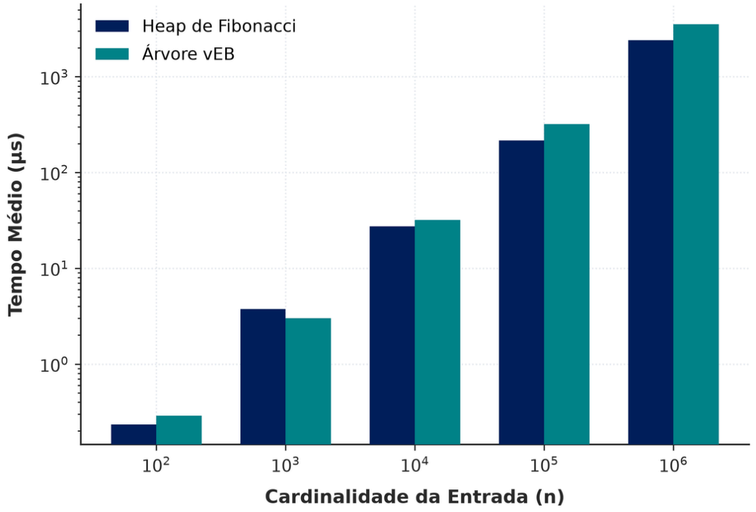
Padrão decrescente



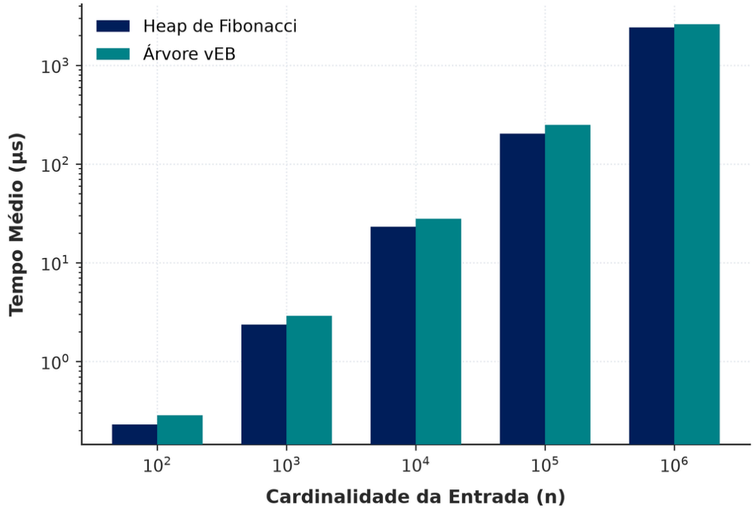
Comparação consulta do mínimo

Tempo médio de **consulta do mínimo**, em µs, no **heap de Fibonacci** e na **árvore vEB** em função de **n**.

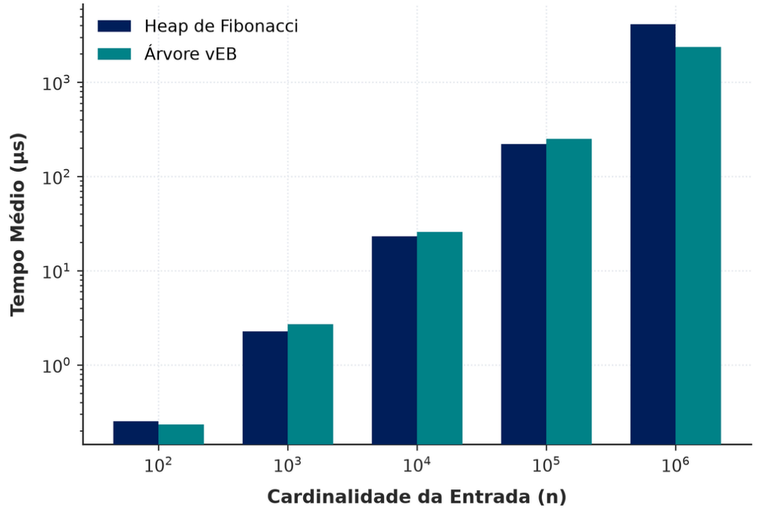
Padrão aleatório



Padrão crescente



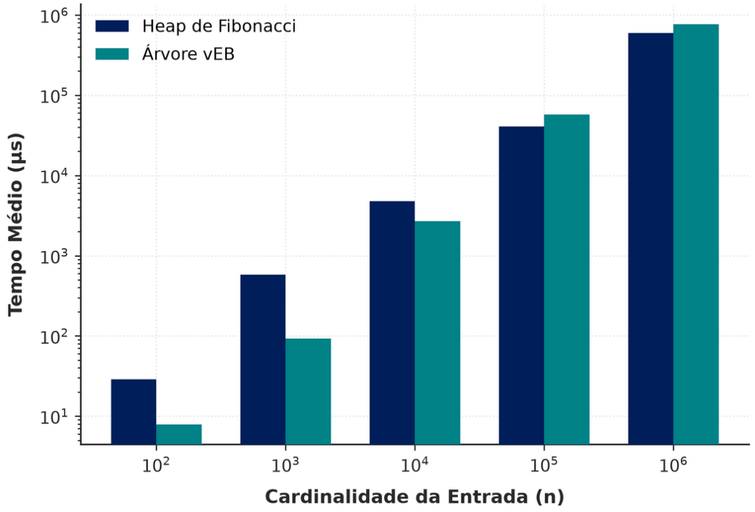
Padrão decrescente



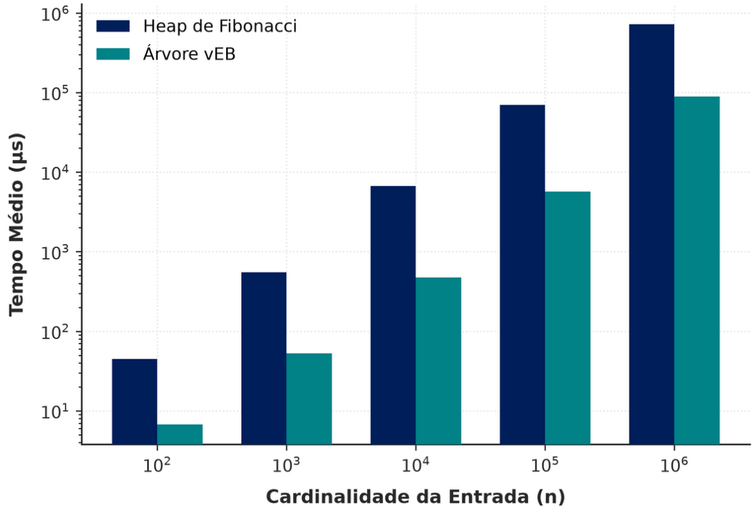
Comparação remoção de elemento específico

Tempo médio de **remoção de elemento específico**, em µs, no **heap de Fibonacci** e na **árvore vEB** em função de **n**.

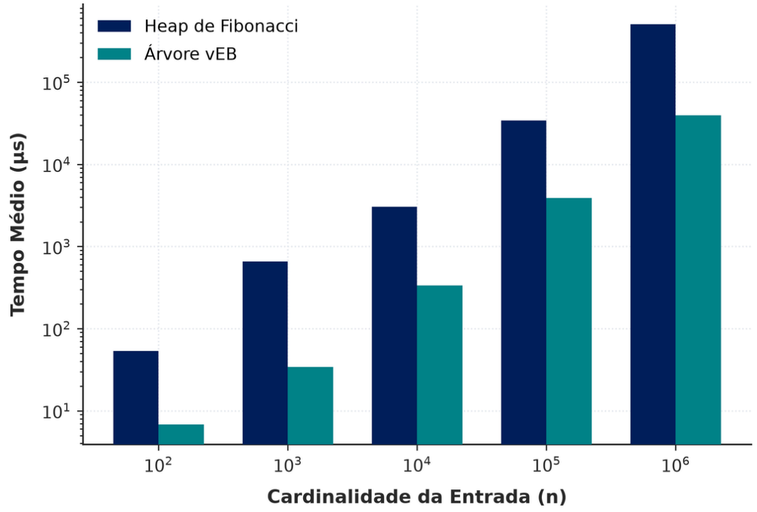
Padrão aleatório



Padrão crescente



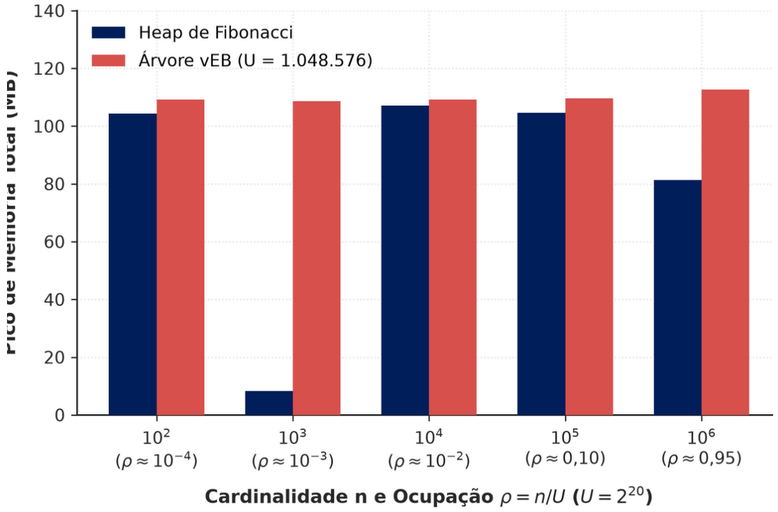
Padrão decrescente



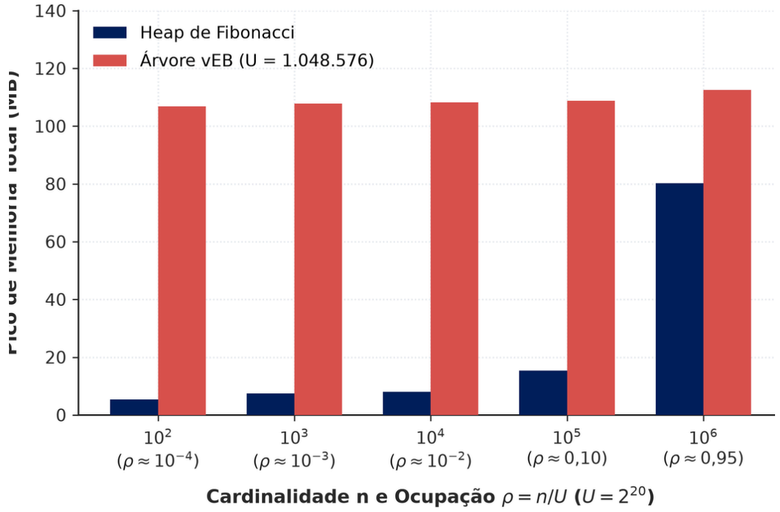
Comparação picos de memória

Pico de memória, em **MB**, do **heap de Fibonacci** e da **árvore vEB** em função de **n**, com universo fixo **U=1.048.576**.

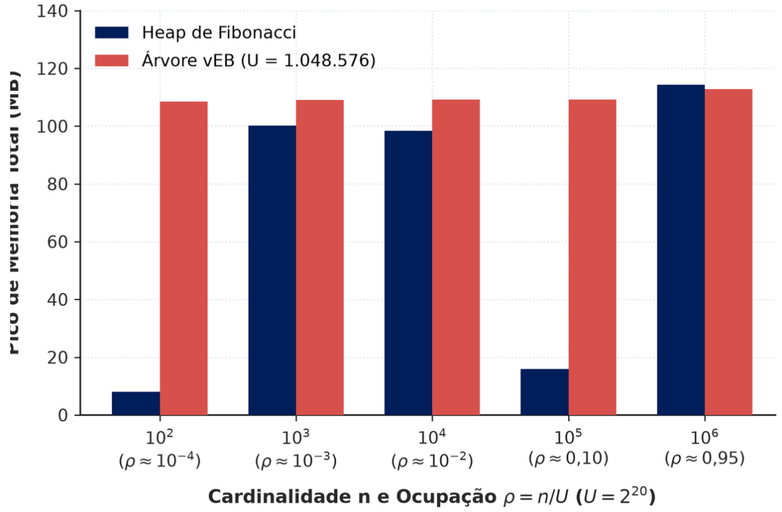
Padrão aleatório



Padrão crescente



Padrão decrescente



Obrigado pela Atenção!

Dúvidas?